UNIWERSYTET WARMIŃSKO MAZURSKI W OLSZTYNIE

WYDZIAŁ MATEMATYKI I INFORMATYKI

Kierunek:
Inżynieria systemów informatycznych

Oleksandr Hrypas

Praca inżynierska

APLIKACJA WEBOWA HR MANAGER

Praca wykonana w Katedrze Informatyki Stosowanej i Modelowania Matematycznego
pod kierunkiem
dr hab. Aleksandra Denisiuka

Olsztyn 2026 r.

1:6247297752

UNIVERSITY OF WARMIA AND MAZURY IN OLSZTYN

FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Field of study
Information Systems Engineering

Oleksandr Hrypas

Engineering thesis

HR MANAGER WEB APPLICATION

Engineering Thesis written in
the Department of Applied Informatics
and Mathematical Modeling
under supervision of
Dr hab. Aleksander Denisiuk

Olsztyn 2026

2:8369970989

Spis treści

Streszczenie 5

Abstract 5

Wykaz skrótów i oznaczeń 6

1. Wstęp 6

2. Cel i zakres pracy 7

3. Analiza wymagań 7

3.1 Wymagania funkcjonalne 7

3.2 Wymagania niefunkcjonalne 7

3.3 Role i uprawnienia 8

3.4 Przypadki użycia i scenariusze biznesowe 8

3.5 Założenia, ograniczenia oraz ryzyka 14

3.6 Kryteria akceptacji 15

3.7 Wymagania bezpieczeństwa i ochrony danych 15

4. Projekt systemu 16

4.1 Architektura systemu 16

4.2 Model danych 17

4.3 Projekt API REST 18

4.4 Projekt interfejsu użytkownika 18

4.5 Projekt bezpieczeństwa i kontroli dostępu 21

4.6 Projekt logiki urlopowej i limitów 21

4.7 Projekt modułu importu obecności 23

4.8 Projekt raportowania i analityki 24

5. Implementacja 25

5.1 Backend – Django i Django REST Framework 25

5.2 Frontend – React i Tailwind CSS 39

6. Testowanie 48

3:2757969244

6.1 Plan testów 49
6.2 Testy API (walidacja backendu) 49
6.3 Testy walidacji wniosków urlopowych 49
6.4 Testy bezpieczeństwa i uprawnień 49
6.5 Testy interfejsu (frontend) 49
6.6 Testy wydajnościowe (podstawowe) 50
6.7 Wnioski z testów 50
7. Instrukcja uruchomienia 50
7.1 Wymagania środowiskowe 50
7.2 Uruchomienie backendu (Windows) 50
7.3 Uruchomienie frontendu 50
7.4 Uwagi dotyczące konfiguracji 51
7.5 Uruchomienie backendu (Linux/macOS) 51
7.6 Budowa wersji produkcyjnej frontendu i integracja z Django 51
8. Instrukcja użytkownika 53
8.1 Role użytkowników 53
8.2 Logowanie i wylogowanie 53
8.3 Panel główny (Dashboard) 53
8.4 Moduł „Pracownicy” (admin/menedżer) 53
8.5 Moduł „Urlopy” 53
8.6 Moduł „Obecności” 54
8.7 Import CSV (admin) 54
8.8 Oceny/Frekwencja i eksport 54
8.9 FAQ 54
9. Podsumowanie i kierunki rozwoju 54
Bibliografia 55

4:7433545661

Streszczenie

Celem pracy było zaprojektowanie i wykonanie aplikacji webowej wspierającej podstawowe procesy zarządzania zasobami ludzkimi w małej organizacji. System „HR Manager” umożliwia ewidencję pracowników, rejestrację obecności (w tym import z pliku CSV), obsługę wniosków urlopowych oraz generowanie miesięcznych podsumowań frekwencji. Aplikacja posiada panel administracyjny Django oraz interfejs użytkownika zbudowany w React z wykorzystaniem stylowania Tailwind CSS. Komunikacja pomiędzy warstwą prezentacji i serwerem odbywa się przez API REST, a dostęp do danych jest chroniony uwierzytelnianiem sesyjnym i tokenowym.
Słowa kluczowe: HR, Django, React, REST API, frekwencja, wnioski urlopowe, import CSV, eksport PDF.

Abstract

The goal of this thesis was to design and implement a web application supporting basic human resources processes in a small organization. The “HR Manager” system enables employee management, attendance tracking (including CSV import), leave request handling, and monthly attendance summaries. The application provides a Django Admin panel and a React-based user interface styled with Tailwind CSS. Communication between the UI and server is performed via a REST API, while data access is protected using session and token authentication.
Keywords: HR, Django, React, REST API, attendance, leave requests, CSV import, PDF export.

5:8239825895

Wykaz skrótów i oznaczeń

W niniejszej pracy stosuje się następujące skróty i oznaczenia:

- API – Application Programming Interface, interfejs programistyczny aplikacji
- CRUD – Create, Read, Update, Delete (operacje na danych)
- SPA – Single Page Application (aplikacja jednostronicowa)
- REST – styl architektury usług sieciowych oparty o zasoby i metody HTTP
- HTTP/HTTPS – protokół komunikacji w sieci / wersja szyfrowana
- CSRF – Cross-Site Request Forgery (atak fałszowania żądań)
- CORS – Cross-Origin Resource Sharing (polityka dostępu między domenami)
- ORM – Object-Relational Mapping (mapowanie obiektów na bazę danych)
- DRF – Django REST Framework, biblioteka do budowy API w Django
- UI/UX – interfejs użytkownika / doświadczenie użytkownika
- RWD – Responsive Web Design (responsywność interfejsu)
- CSV – format tekstowy danych rozdzielanych przecinkami
- XLSX – format arkusza kalkulacyjnego Microsoft Excel
- PDF – Portable Document Format
- RBAC – Role Based Access Control (kontrola dostępu wg ról)
- KPI – Key Performance Indicator (wskaźniki efektywności)

1. Wstęp

Procesy kadrowe w małych i średnich firmach są często realizowane w arkuszach kalkulacyjnych lub w kilku rozproszonych narzędziach. Prowadzi to do niespójności danych, utrudnia raportowanie oraz zwiększa ryzyko błędów. Celem projektu „HR Manager” jest integracja najważniejszych czynności działu HR w jednym systemie: ewidencji pracowników, obsługi urlopów, rejestru obecności oraz zestawień miesięcznych.

6:9346941255

2. Cel i zakres pracy

Celem pracy jest stworzenie aplikacji webowej, która umożliwia centralne zarządzanie danymi kadrowymi oraz analizę frekwencji na podstawie rejestru obecności. Zakres pracy obejmuje:

• zaprojektowanie modelu danych i architektury systemu (frontend SPA + backend REST);

• implementację API w Django REST Framework wraz z mechanizmami uprawnień; [2]

• implementację interfejsu użytkownika w React (formularze, tabele, wykresy);

• import obecności z pliku CSV oraz eksport raportów do CSV/XLSX/PDF;

• opracowanie instrukcji uruchomienia i scenariuszy testowych.

3. Analiza wymagań

3.1 Wymagania funkcjonalne

• Logowanie użytkownika oraz wylogowanie.

• Zarządzanie pracownikami (CRUD) z wyszukiwaniem i paginacją listy.

• Tworzenie wniosków urlopowych (daty, powód) oraz podgląd historii wniosków.

• Akceptacja/odrzucenie wniosków urlopowych przez użytkownika z rolą menedżera/administratora.

• Rejestr obecności dziennej (status, liczba minut pracy, notatki).

• Import danych obecności z pliku CSV (kolumny: email, date, status, minutes, notes).

• Generowanie miesięcznych podsumowań frekwencji (oceny) na podstawie danych obecności.

• Eksport podsumowań do CSV, XLSX oraz PDF.

• Panel administracyjny Django do wglądu i zarządzania danymi (dla administratora).

3.2 Wymagania niefunkcjonalne

• Responsywny i czytelny interfejs (ciemny motyw, spójne komponenty).

• Obsługa dużej liczby rekordów przez paginację oraz przewijanie tabel.

• Walidacja danych po stronie serwera (spójność dat, zakresy wartości, integralność danych).

7:2473784897

• Bezpieczeństwo: dostęp do API tylko dla użytkowników uwierzytelnionych; rozróżnienie uprawnień (admin vs użytkownik).

• Możliwość uruchomienia w środowisku lokalnym Windows bez usług zewnętrznych (SQLite).

3.3 Role i uprawnienia

W systemie wyróżniono role logiczne, wynikające z uprawnień w Django:

• Użytkownik (pracownik) – tworzy własne wnioski urlopowe i przegląda własne dane.

• Menedżer/Administrator – przegląda dane wszystkich pracowników, akceptuje lub odrzuca wnioski urlopowe oraz zarządza wpisami.

W panelu administracyjnym Django widoczne są również standardowe modele dostarczane przez Django oraz zainstalowane biblioteki (np. django-allauth: konta społecznościowe, tokeny). Nie są one wykorzystywane bezpośrednio w logice aplikacji, ale pozostają dostępne w ramach konfiguracji.

3.4 Przypadki użycia i scenariusze biznesowe

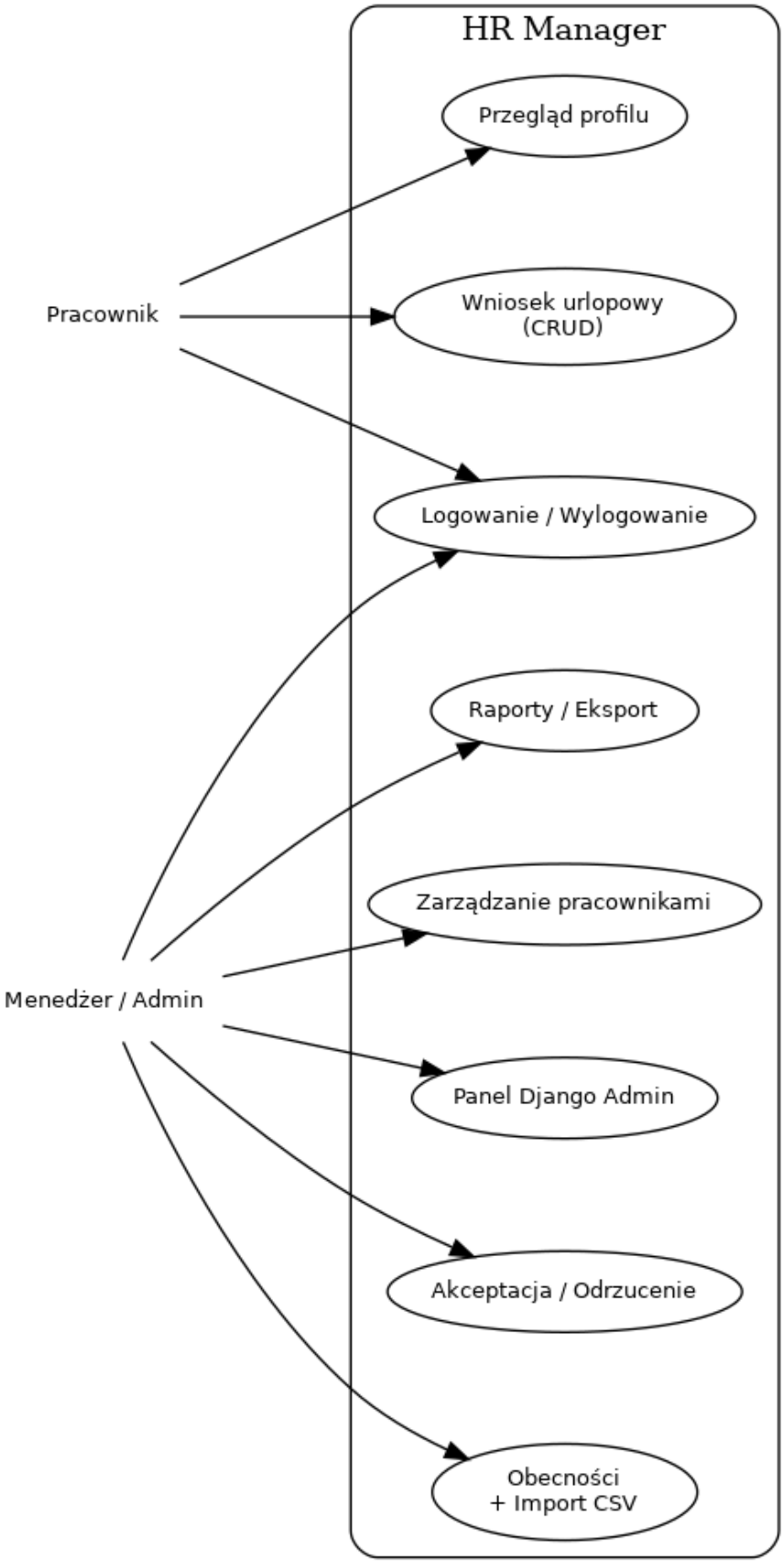
System „HR Manager" wspiera podstawowe procesy kadrowe w organizacji poprzez zestaw przypadków użycia. Poniżej przedstawiono najważniejsze scenariusze wraz z opisem przebiegu oraz spodziewanym rezultatem.

UC-01: Logowanie użytkownika

1. Użytkownik otwiera aplikację w przeglądarce.
2. Wprowadza adres e-mail i hasło.
3. System weryfikuje dane i ustanawia sesję.
4. Użytkownik zostaje przekierowany do panelu głównego.

Rezultat: użytkownik uzyskuje dostęp do funkcji zgodnych z rolą (pracownik/menedżer/admin).

8:5462178039

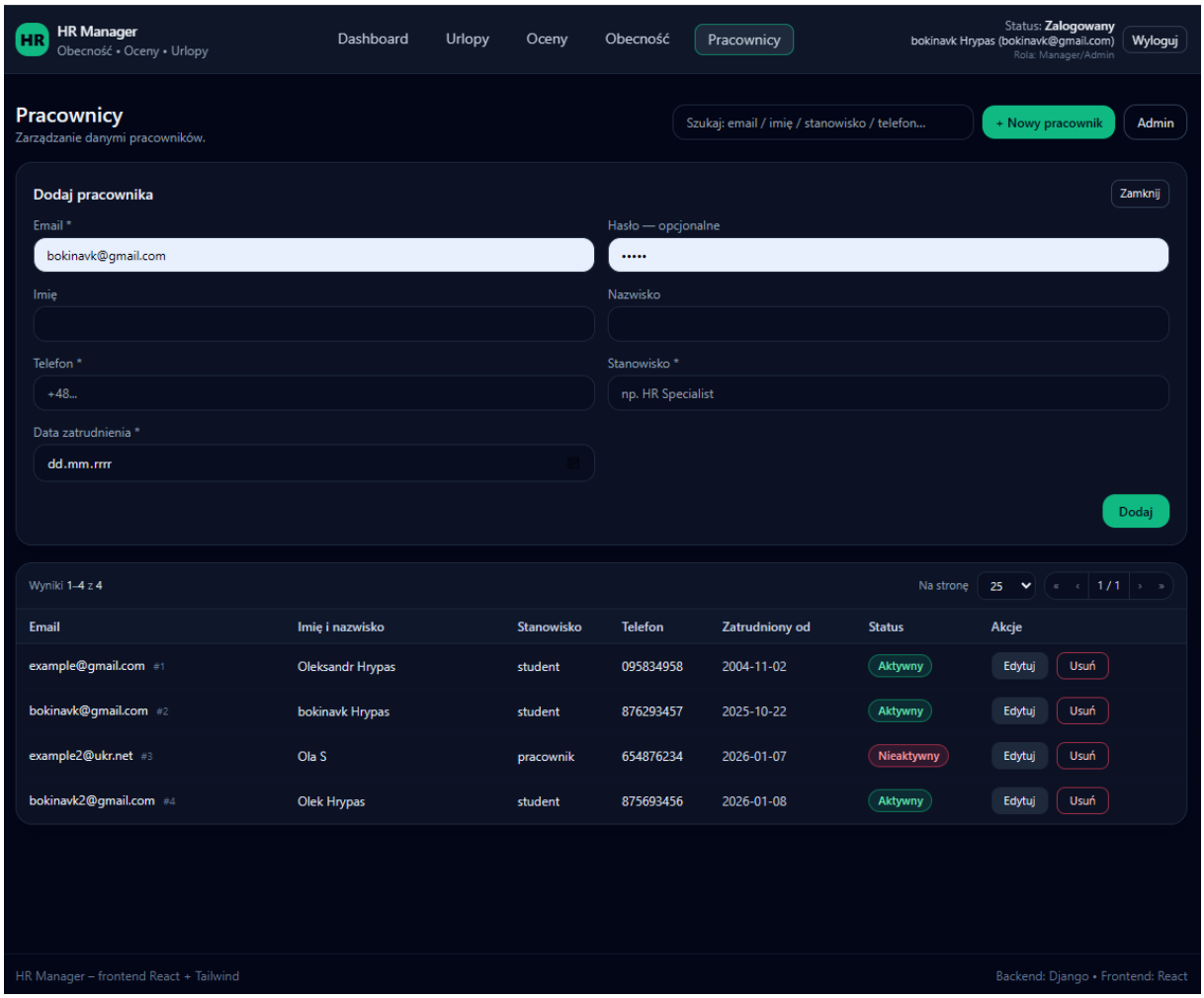


Rys. 3.1. Diagram przypadków użycia (UML) – logowanie oraz moduły systemu.

UC-02: Utworzenie profilu pracownika (rola: admin/menedżer)

1. Administrator przechodzi do modułu „Pracownicy”.
2. Wybiera opcję „Dodaj pracownika”.
3. Wprowadza dane (e-mail, imię, nazwisko, stanowisko, data zatrudnienia, telefon).
4. System tworzy konto użytkownika oraz encję Employee.

Rezultat: pracownik pojawia się na liście i może zalogować się do systemu.

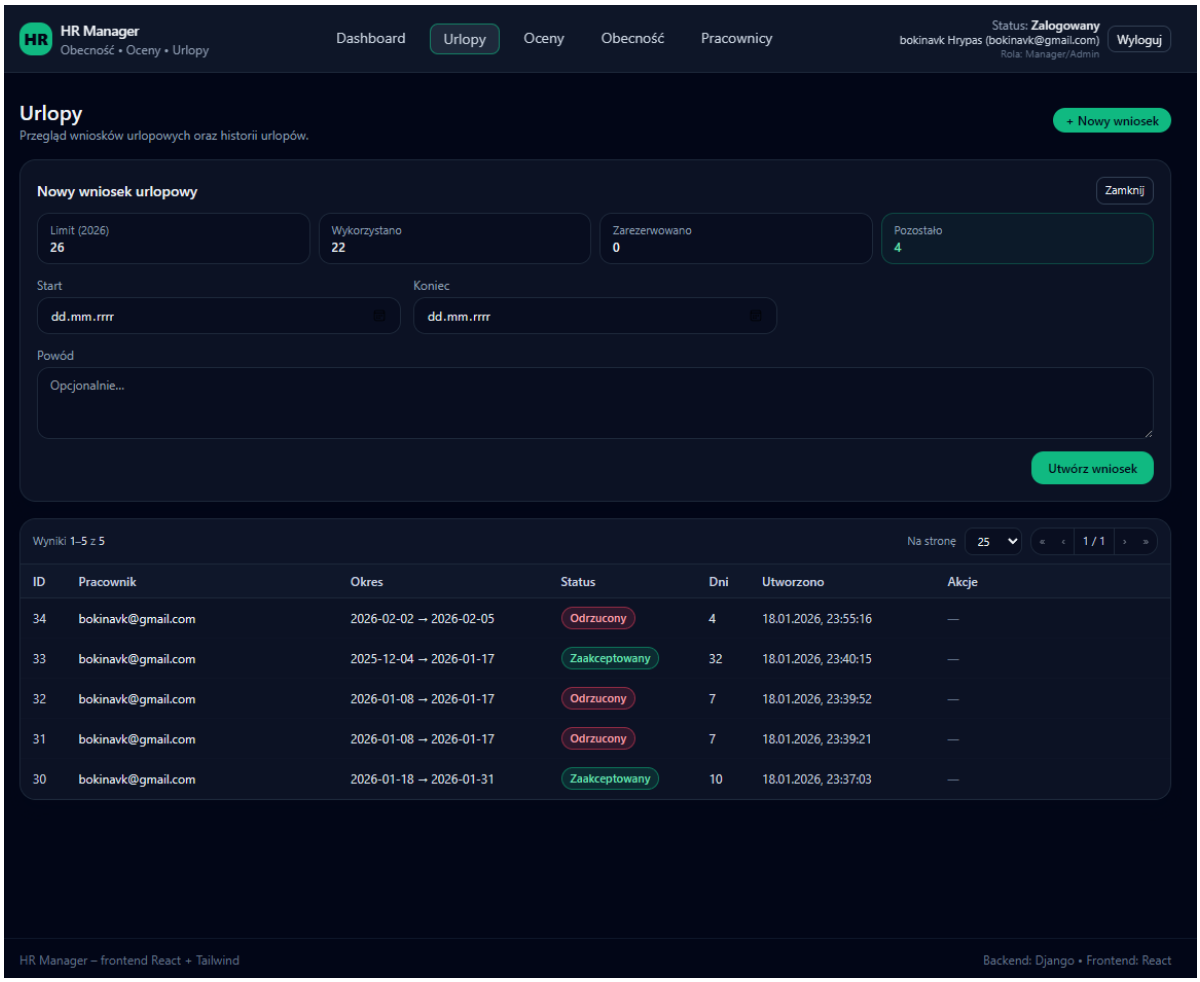


Rys. 3.2. Formularz dodawania pracownika.

UC-03: Złożenie wniosku urlopowego (rola: pracownik)

1. Pracownik wybiera moduł „Urlopy”.
2. Ustawia datę rozpoczęcia i zakończenia oraz (opcjonalnie) uzasadnienie.
3. System waliduje poprawność zakresu dat, liczbę dni roboczych i limit roczny.
4. System zapisuje wniosek ze statusem „pending”.

Rezultat: wniosek oczekuje na decyzję.

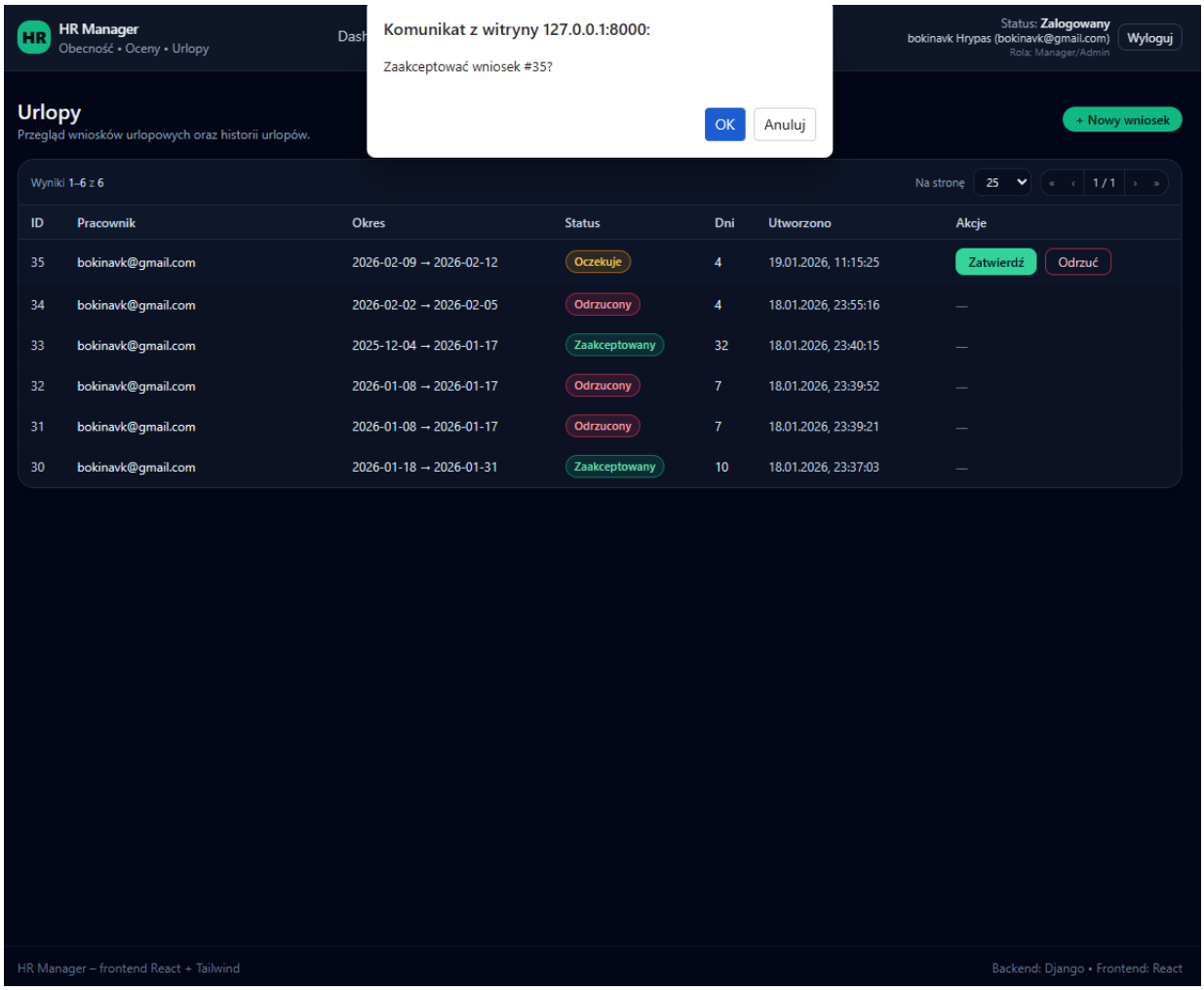


Rys. 3.3. Widok listy wniosków urlopowych + statusy.

UC-04: Akceptacja/odrzucenie wniosku (rola: menedżer/admin)

1. Menedżer otwiera listę wniosków „Oczekujące”.
2. Wybiera wniosek i podejmuje decyzję (approve/reject).
3. System ponownie uruchamia walidację konfliktów i limitów (dla approve).
4. System zmienia status i zapisuje decyzję.

Rezultat: status wniosku zmienia się na „approved” lub „rejected”.

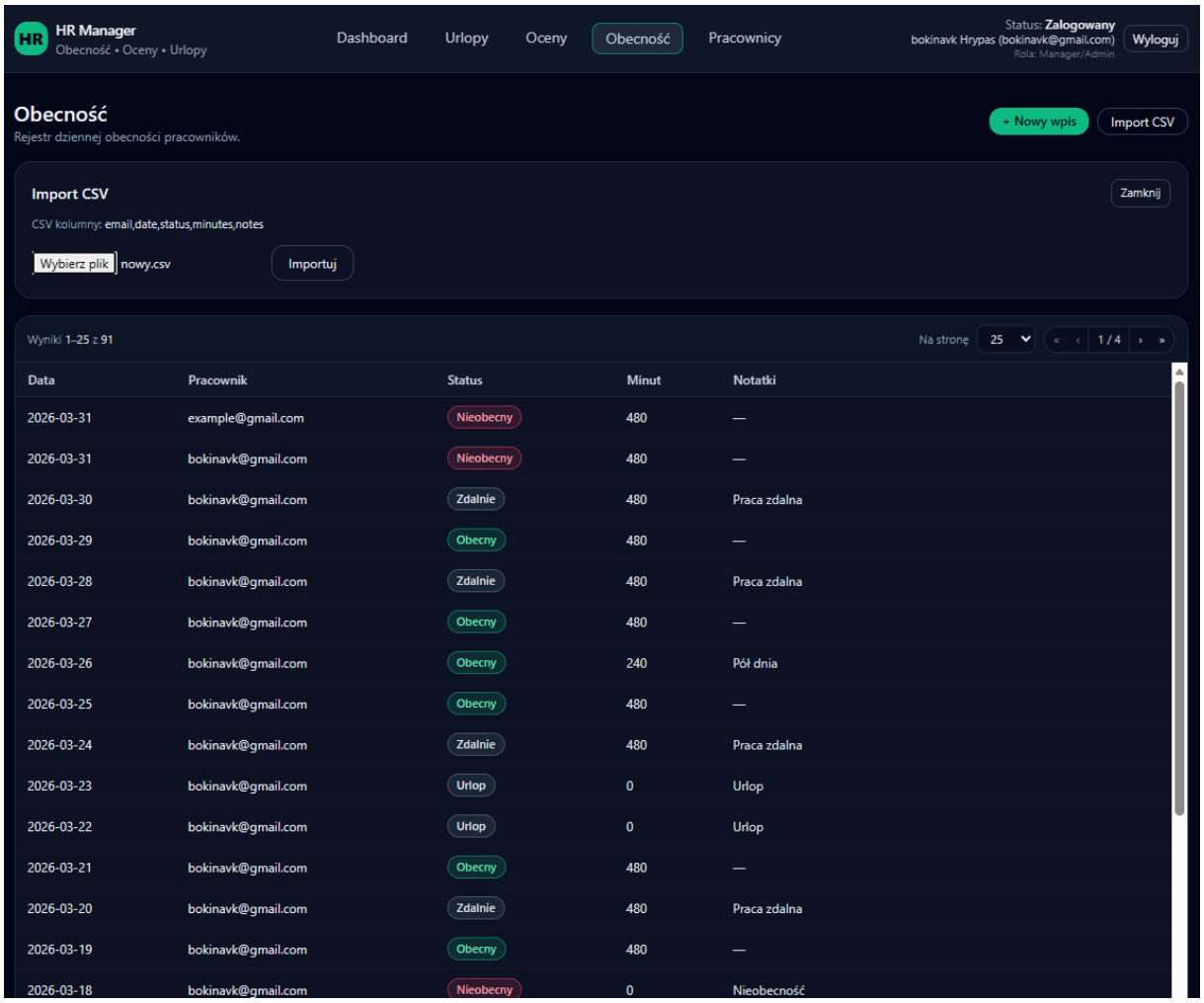


Rys. 3.4. Panel akceptacji wniosków urlopowych.

UC-05: Import obecności z pliku CSV (rola: admin)

1. Administrator przechodzi do „Obecności” i wybiera import pliku.
2. System wczytuje CSV, waliduje format i rekordy.
3. Dla każdego wiersza tworzy lub aktualizuje rekord obecności.
4. System przelicza podsumowania miesięczne (PerformanceReview) i zwraca raport importu.

Rezultat: obecności są zasilone danymi, a system zwraca liczbę rekordów utworzonych, zaktualizowanych i błędnych.

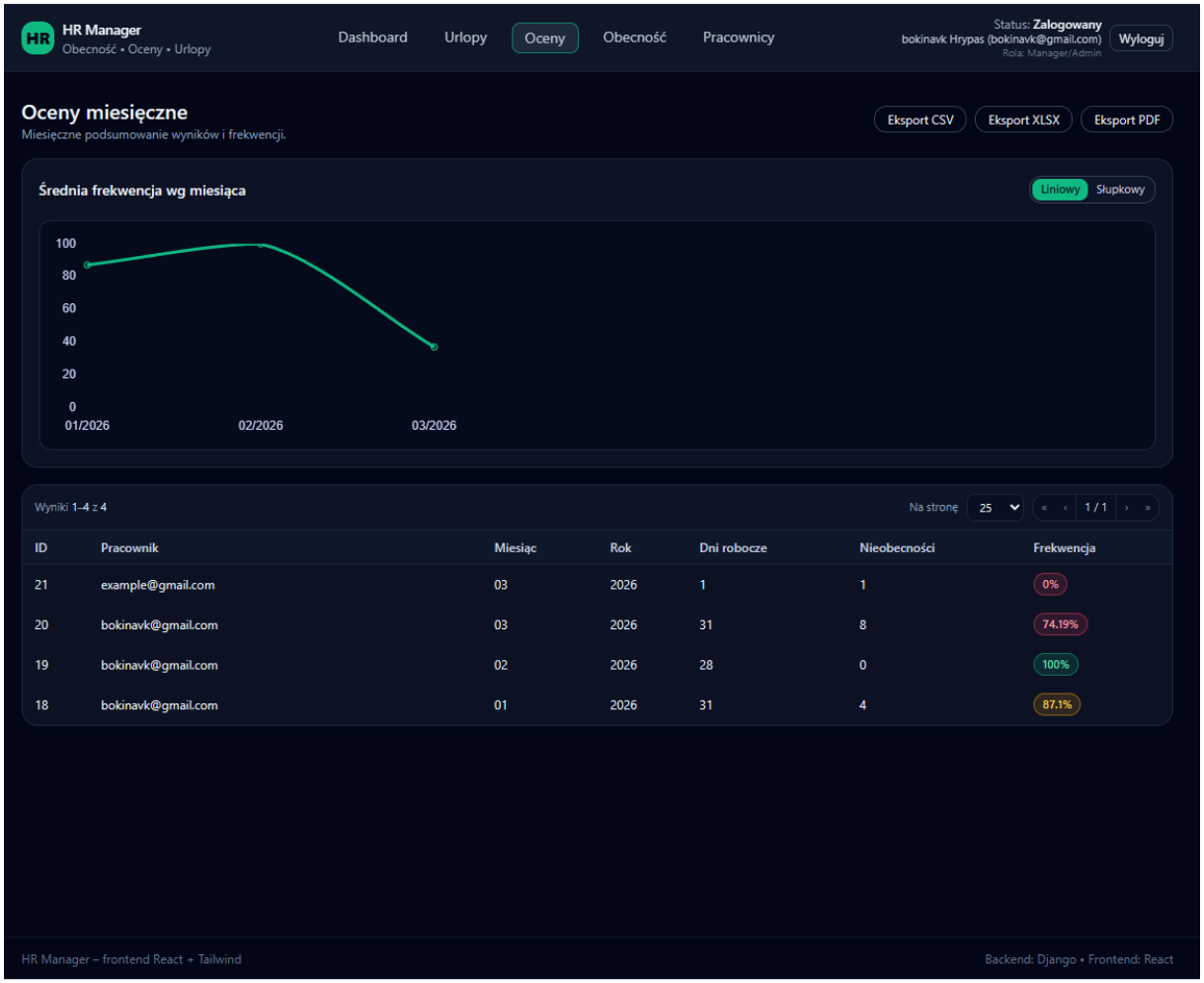


Rys. 3.5. Widok importu CSV.

UC-06: Eksport raportu ocen miesięcznych (rola: admin/menedżer)

1. Użytkownik wybiera moduł „Oceny/Frekwencja”.
2. Określa format pliku (CSV/XLSX/PDF).
3. System generuje plik na podstawie widocznych danych.
4. Plik zostaje pobrany przez przeglądarkę.

Rezultat: użytkownik otrzymuje raport w wybranym formacie.



Rys. 3.6. Wykres frekwencji i przyciski eksportu.

3.5 Założenia, ograniczenia oraz ryzyka

Założenia projektowe:

• system jest aplikacją webową (frontend SPA + backend REST API);

• użytkownicy logują się do systemu kontem wbudowanym (sesje/cookies);

• dane są przechowywane w relacyjnej bazie danych (SQLite w wersji lokalnej);

• import danych obecności zakłada spójny format CSV.

Ograniczenia:

• brak integracji z zewnętrznymi systemami HR oraz brak automatycznych powiadomień e-mail;

• brak kalendarza świąt państwowych (dni robocze liczone jako pon–pt);

• brak wieloetapowego workflow zatwierdzania (w wersji demonstracyjnej).

Ryzyka i sposoby minimalizacji:

• ryzyko błędnych danych z importu – walidacja rekordów, raport błędów i kontynuacja importu;

• ryzyko konfliktów urlopów – walidacja nakładających się zakresów dat po stronie serwera;

• ryzyko naruszenia dostępu do danych – kontrola uprawnień na poziomie endpointów;

• ryzyko utraty danych – zalecenia kopii zapasowej bazy i eksportów.

3.6 Kryteria akceptacji

Za kryteria akceptacji przyjęto spełnienie poniższych warunków:

1. System umożliwia logowanie i wylogowanie użytkownika oraz ogranicza dostęp do API wyłącznie do użytkowników uwierzytelnionych.
2. Administrator może tworzyć i edytować pracowników oraz ich dane profilowe.
3. Pracownik może utworzyć wniosek urlopowy, a system weryfikuje poprawność zakresu dat, co najmniej 1 dzień roboczy, brak konfliktu oraz limit urlopu w każdym roku.
4. Menedżer/admin może akceptować lub odrzucać wnioski.
5. System umożliwia rejestrację obecności oraz import CSV z raportem błędów.
6. System automatycznie generuje miesięczne podsumowania frekwencji i umożliwia eksport CSV/XLSX/PDF.
7. Interfejs działa poprawnie w typowych rozdzielczościach (RWD) i zapewnia czytelność treści.

3.7 Wymagania bezpieczeństwa i ochrony danych

System przetwarza dane osobowe pracowników (m.in. e-mail, imię, nazwisko, telefon, historia urlopów i obecności), dlatego zastosowano mechanizmy uwierzytelnienia, kontroli dostępu oraz walidacji danych wejściowych.

• Uwierzytelnienie i autoryzacja: dostęp do zasobów ograniczony do ról; pracownik widzi własne dane, a admin/menedżer dane globalne. [13]

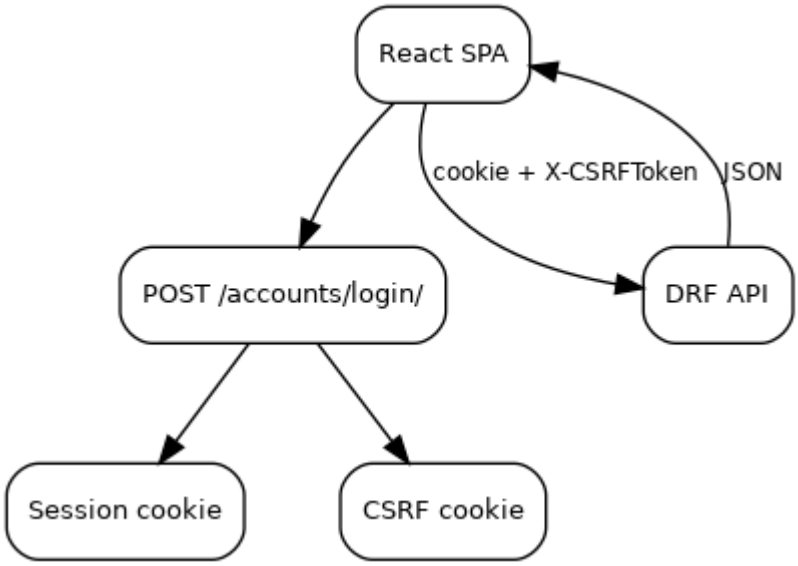
• Ochrona CSRF: żądania modyfikujące dane (POST/PUT/PATCH/DELETE) wymagają tokenu CSRF w SPA.

• Walidacja danych wejściowych: walidacja serwerowa w serializerach i modelach (constrainty, zakresy, długości).

15:2335605074

• Minimalizacja uprawnień: endpointy administracyjne dostępne wyłącznie dla użytkowników z uprawnieniami.

• Rozszerzenie: możliwość dodania logu audytowego (kto i kiedy zmienił dane).

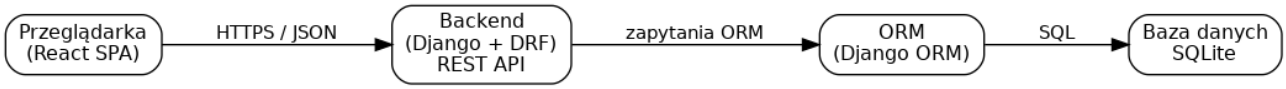


Rys. 3.7. Schemat przepływu uwierzytelnienia (sesja + CSRF) – diagram.

4. Projekt systemu

4.1 Architektura systemu

Aplikacja została zaprojektowana w architekturze klient-serwer. Warstwa prezentacji jest realizowana jako SPA (Single Page Application) w React, natomiast backend stanowi aplikacja Django udostępniająca REST API. Dane są przechowywane w relacyjnej bazie SQLite. Komunikacja odbywa się przez HTTP/HTTPS w formacie JSON.



Rys. 4.1. Diagram architektury (React -> REST API -> ORM -> SQLite).

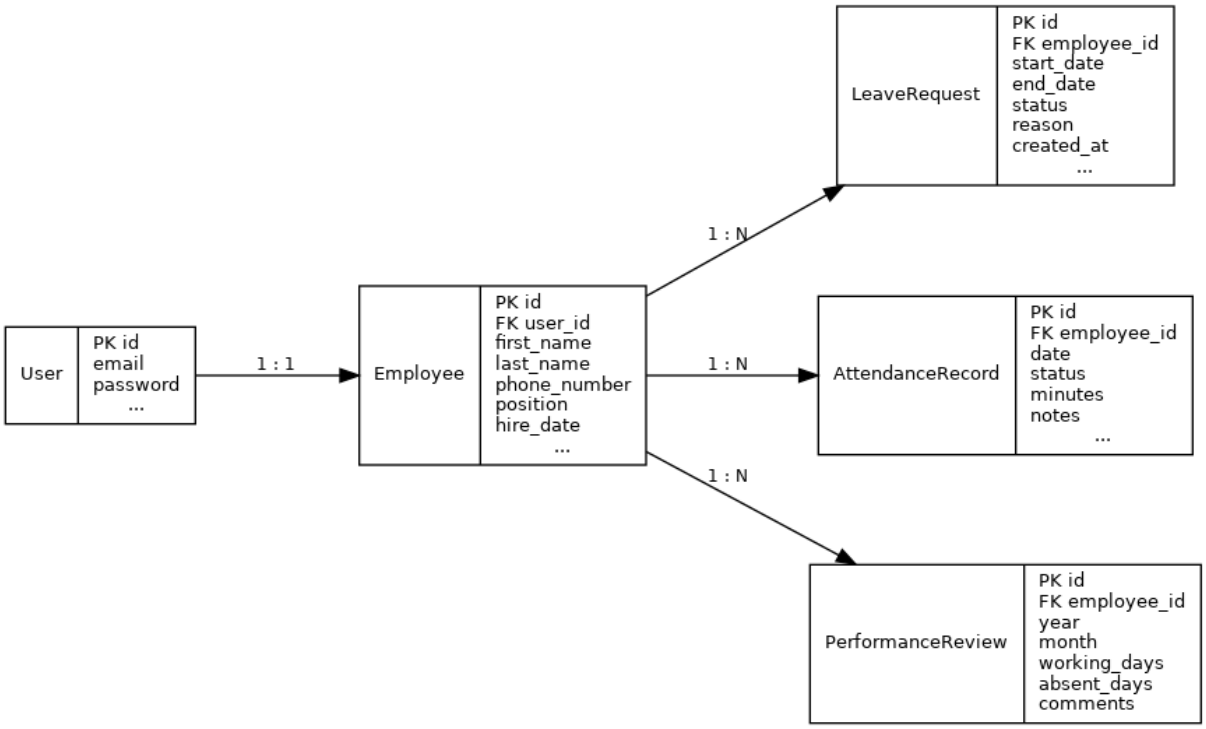
16:8192358147

4.2 Model danych

Tabela 4.1 przedstawia główne encje bazy danych oraz ich przeznaczenie.

Tabela 4.1. Główne encje systemu.

Encja	Najważniejsze pola	Opis
Employee	user, phone_number, position, hire_date	Dane pracownika powiązane z kontem użytkownika. (Pola first_name, last_name, is_active przechowywane są w modelu User (Django))
LeaveRequest	employee, start_date, end_date, reason, status, created_at	Wniosek urlopowy ze statusem oraz walidacją dat.
AttendanceRecord	employee, date, status, minutes_worked, notes	Rejestr obecności dziennej wraz z notatką.
PerformanceReview	employee, year, month, working_days, absent_days, comments	Miesięczne podsumowanie frekwencji oraz komentarz.



Rys. 4.2. Diagram ERD bazy danych (relacje między Employee, LeaveRequest, AttendanceRecord, PerformanceReview).

17:1052344156

4.3 Projekt API REST

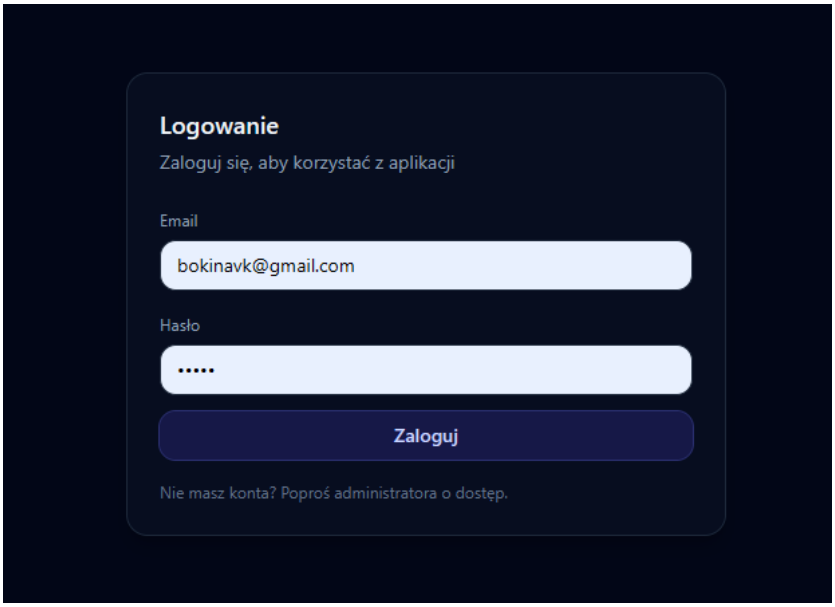
Tabela 4.2 zawiera zestawienie kluczowych endpointów API (prefiks /api/).

Tabela 4.2. Zestawienie endpointów API.

Zasób	Metody	Dostęp	Opis
employees/	GET, POST, PUT, DELETE	admin/manager	Lista i zarządzanie pracownikami.
leave-requests/	GET, POST, PUT, DELETE	user/admin	Wnioski urlopowe (user widzi swoje, admin wszystkie).
leave-requests/{id}/approve/	POST	admin/manager	Akceptacja wniosku urlopowego.
leave-requests/{id}/reject/	POST	admin/manager	Odrzucenie wniosku urlopowego.
attendance/	GET, POST, PUT, DELETE	admin/manager	Rejestr obecności.
attendance/import_csv/	POST	admin/manager	Import obecności z CSV.
performance/	GET, POST, PUT, DELETE	admin/manager	Podsumowania miesięczne.
performance/export/?file=csv|xlsx|pdf	GET	admin/manager	Eksport raportów.
me/	GET	zalogowany	Dane aktualnego użytkownika (nagłówek UI).

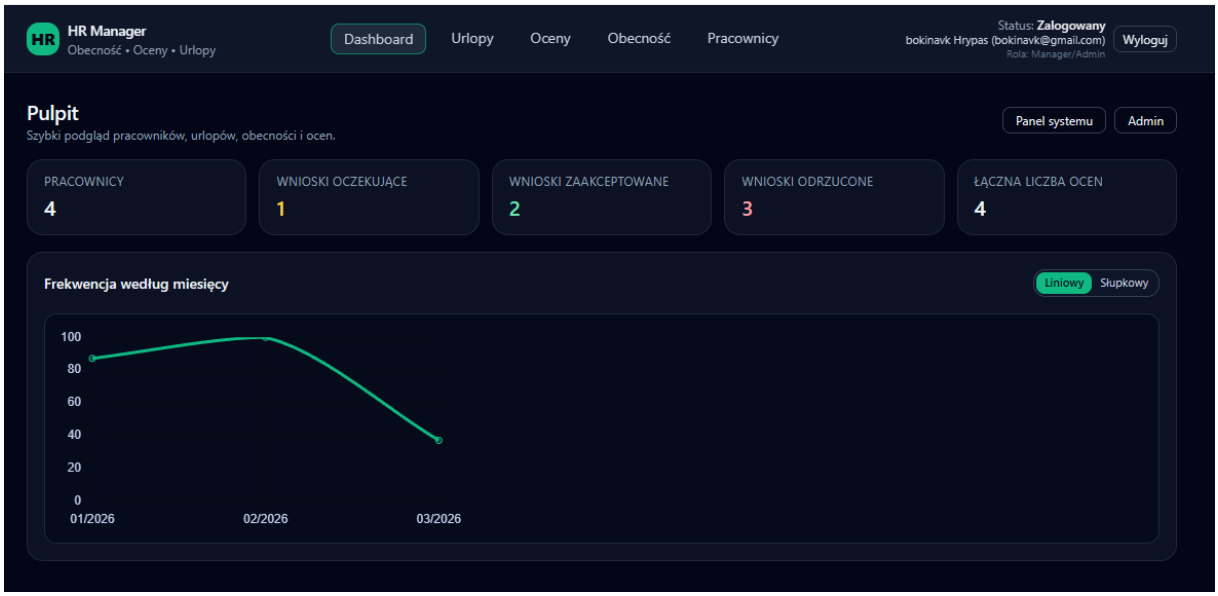
4.4 Projekt interfejsu użytkownika

Interfejs użytkownika udostępnia nawigację górną do modułów: Pulpit, Urlopy, Oceny, Obecność oraz Pracownicy. W systemie zastosowano spójny ciemny motyw oraz tabele z paginacją. Poniżej przedstawiono wybrane ekrany aplikacji.

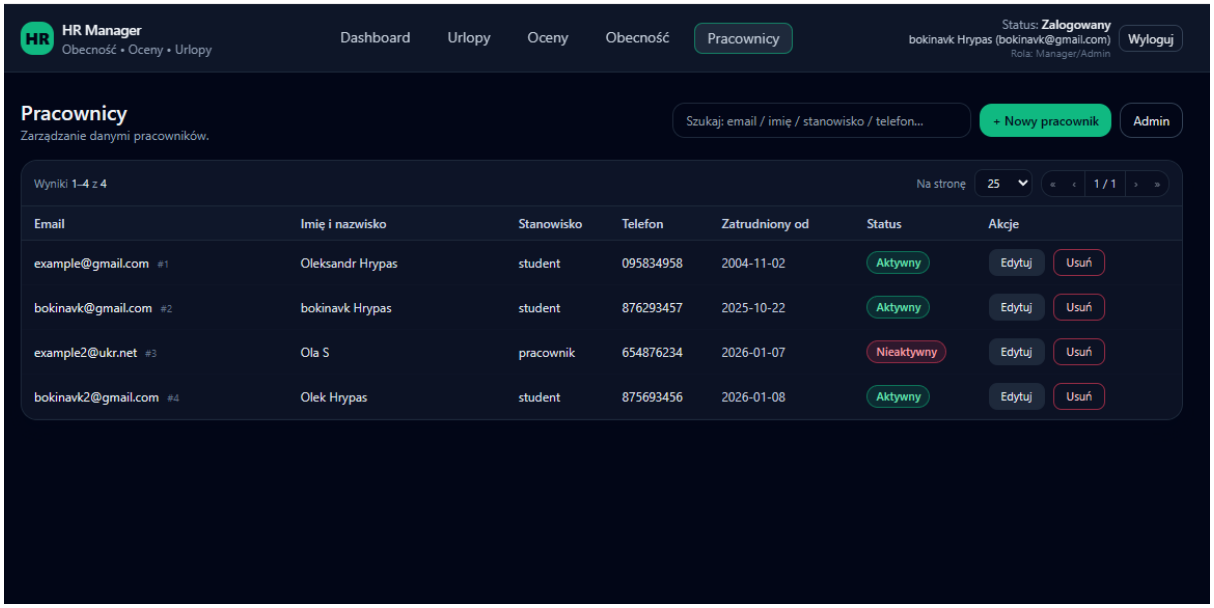


Rys. 4.3. Ekran logowania użytkownika.

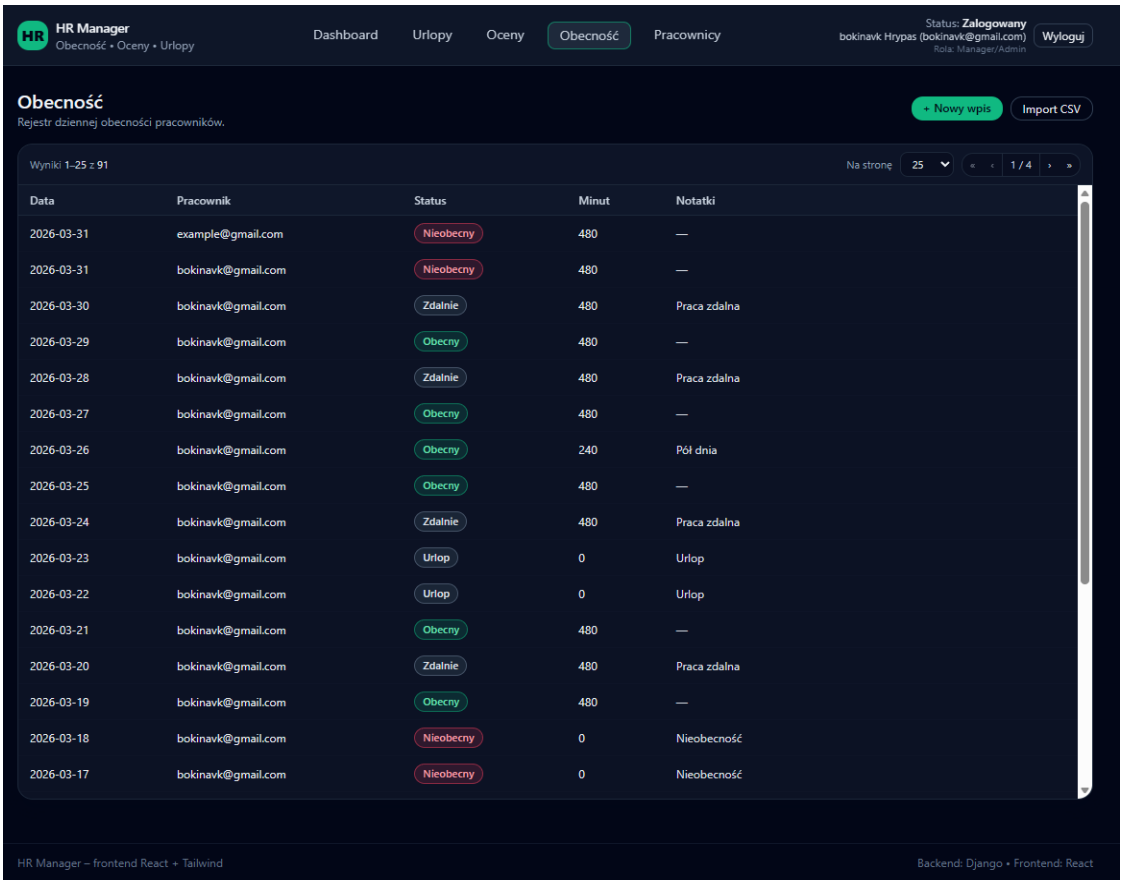
18:9473685354



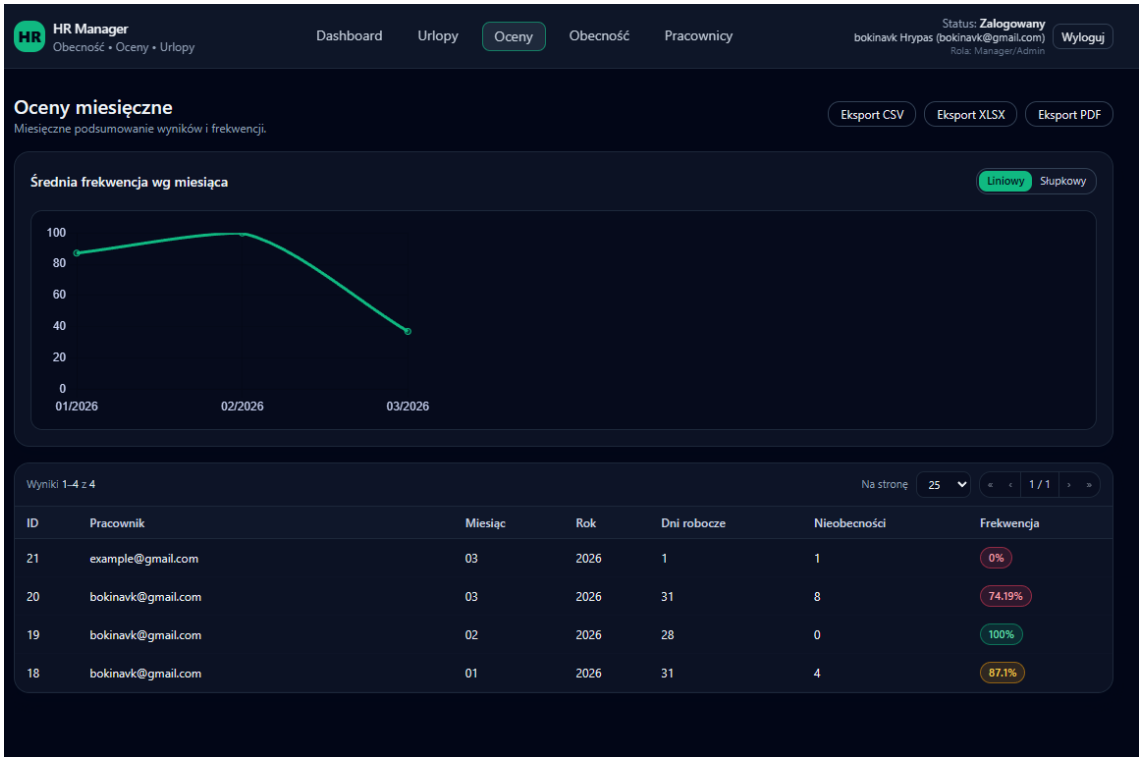
Rys. 4.4. Pulpit (Dashboard) – podsumowania i wykres.



Rys. 4.5. Lista pracowników z paginacją i akcjami CRUD.



Rys. 4.6. Rejestr obecności – tabela z przewijaniem.



Rys. 4.7. Oceny miesięczne – wykres i tabela wyników.

4.5 Projekt bezpieczeństwa i kontroli dostępu

W projekcie przyjęto model kontroli dostępu oparty o role użytkowników. W warstwie backendu mechanizmy bezpieczeństwa realizowane są przez uwierzytelnienie sesyjne (cookies), mechanizmy uprawnień DRF oraz filtrowanie danych po stronie serwera.

Dostęp do danych jest ograniczony zgodnie z zasadą minimalnych uprawnień: pracownik widzi wyłącznie informacje związane z własnym profilem, natomiast administrator i menedżer mają wgląd w dane całej organizacji.

Listing 4.1. Fragment klasy uprawnień (permission) – kontrola admin vs read-own.

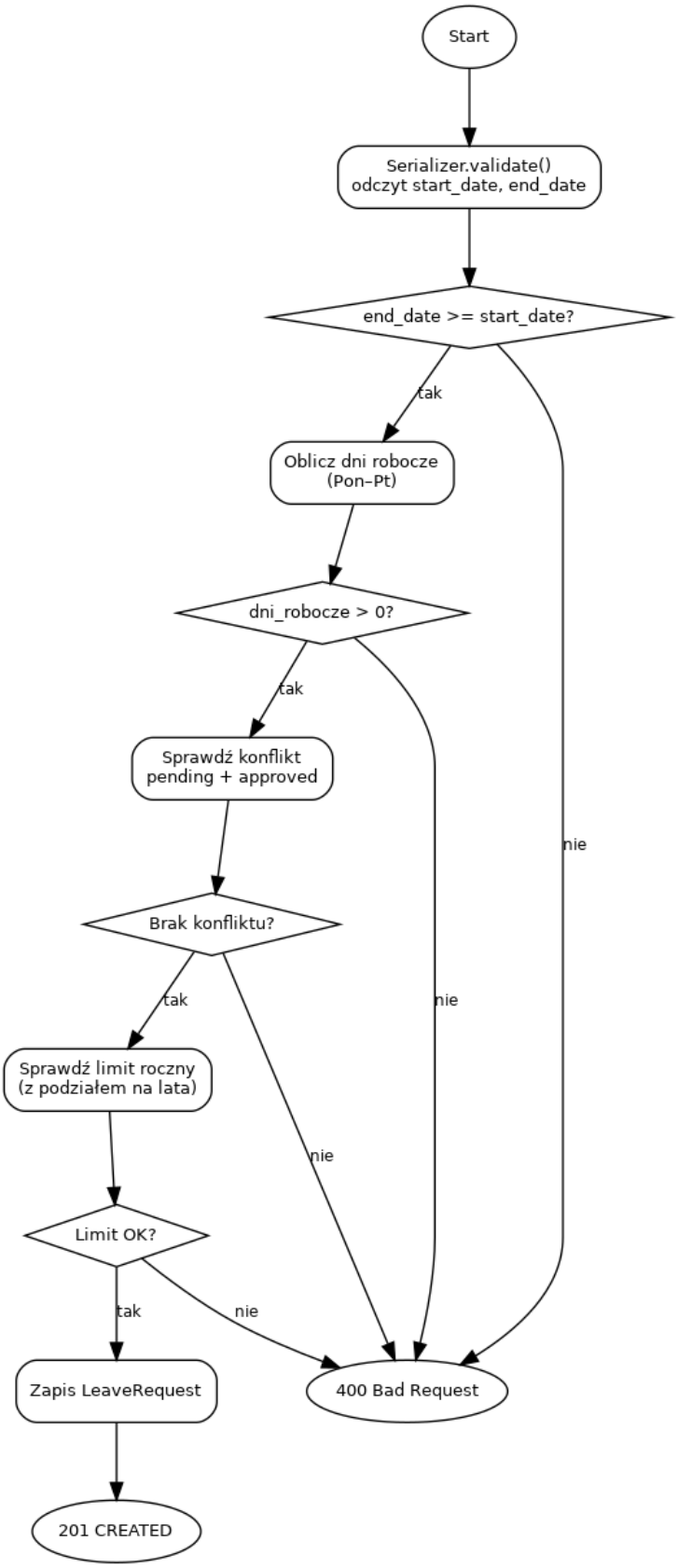
```
class IsAdminOrReadOwn(permissions.BasePermission):

def has_permission(self, request, view):
    return bool(request.user and request.user.is_authenticated)

def has_object_permission(self, request, view, obj):
    if request.user.is_staff or request.user.is_superuser:
        return True

    if isinstance(obj, Employee):
        return obj.user_id == request.user.id

    employee = getattr(obj, "employee", None)
    if isinstance(employee, Employee):
        return employee.user_id == request.user.id

    return False
```

4.6 Projekt logiki urlopowej i limitów

Moduł urlopów wymaga spójnych reguł biznesowych. Urlop liczony jest w dniach roboczych (poniedziałek–piątek), a wniosek nie może kolidować z innym wnioskiem (pending/approved). Dodatkowo kontrolowany jest limit roczny (domyślnie 26 dni), liczony osobno dla każdego roku w przypadku zakresu obejmującego przełom roku.

21:6767492143

Walidacja została umieszczona po stronie serwera, dzięki czemu dotyczy zarówno tworzenia/edycji, jak i akceptacji wniosku.

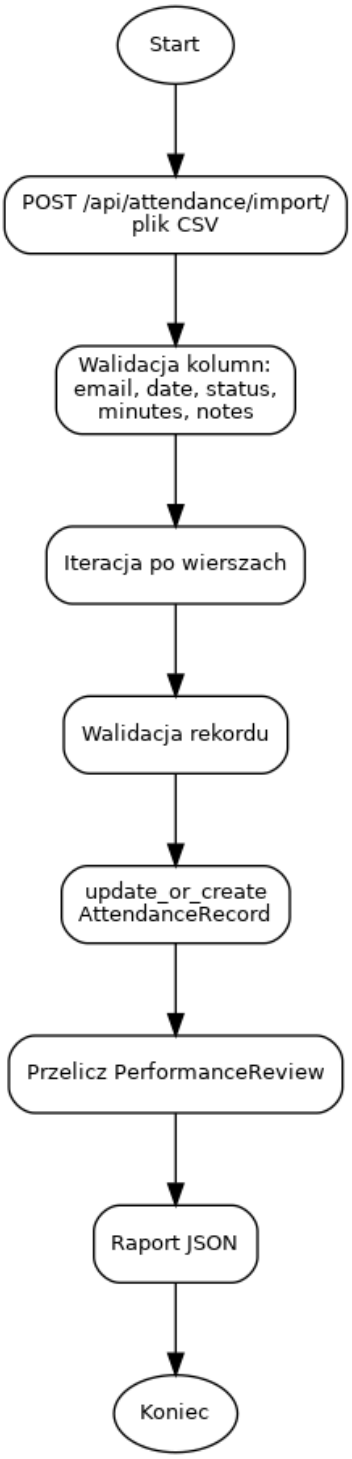


Rys. 4.8. Schemat walidacji wniosku urlopowego – diagram blokowy.

22:7973651643

4.7 Projekt modułu importu obecności

Import CSV zaprojektowano jako proces odporny na błędy: każdy wiersz pliku jest walidowany niezależnie, a błędne rekordy są raportowane bez przerywania całego importu. Dane są zapisywane w trybie create/update, co umożliwia ponowne wgrywanie plików bez ryzyka duplikacji.



Rys. 4.9. Diagram procesu importu CSV (AttendanceRecord) – diagram.

4.8 Projekt raportowania i analityki

Raportowanie obejmuje podsumowania miesięczne (PerformanceReview) oraz wskaźnik frekwencji liczony na podstawie danych obecności. Użytkownik może pobrać raport w formatach CSV (analiza danych), XLSX (praca biznesowa) oraz PDF (prezentacja).

Performance Reviews

ID	Email	Rok	Mies.	Robocze	Nieob.	Frekw.%	Komentarz
21	example@gmail.com	2026	3	1	1	0.0	
20	bokinavk@gmail.com	2026	3	31	8	74.19	
19	bokinavk@gmail.com	2026	2	28	0	100.0	
18	bokinavk@gmail.com	2026	1	31	4	87.1	

Rys. 4.10. Przykładowy układ raportu PDF – zrzut ekranu.

4.9 Projekt UI – ergonomia i dostępność

Interfejs użytkownika zaprojektowano w sposób spójny i minimalistyczny: jednolita nawigacja, tabele z paginacją i wyszukiwaniem oraz czytelne oznaczenia statusów. Zastosowano zasady dostępności, m.in. logiczną kolejność pól formularzy, komunikaty walidacyjne oraz responsywność (RWD).

4.10 Model danych i struktura bazy danych

Warstwa danych oparta jest o relacyjną bazę danych. Model został zaprojektowany w oparciu o encje: użytkownik, pracownik, wniosek urlopowy, rekord obecności oraz podsumowanie miesięczne.

W projekcie zastosowano indeksy oraz ograniczenia integralności, m.in. unikalność rekordu obecności dla pary (pracownik, data) oraz ograniczenia na zakresy i statusy. Podejście to ułatwia utrzymanie spójności danych.

24:5151200964

5. Implementacja

5.1 Backend – Django i Django REST Framework

Backend został zaimplementowany w Python 3.12.10 z użyciem Django 4.2+ oraz Django REST Framework. Domyślne uwierzytelnianie dla API to SessionAuthentication oraz TokenAuthentication, a domyślne uprawnienia wymagają zalogowania (IsAuthenticated). [1] [2]

5.1.1 Konfiguracja projektu (hr_manager/settings.py)

W pliku settings.py skonfigurowano: aplikacje INSTALLED_APPS (w tym rest_framework, allauth, corsheaders), middleware (obsługa CORS i sesji), ustawienia REST_FRAMEWORK oraz statyczne pliki frontendu. W środowisku produkcyjnym zalecane jest przeniesienie SECRET_KEY do zmiennej środowiskowej (.env). [7] [8]

5.1.2 Modele danych (core/models.py)

Modele odpowiadają za strukturę bazy danych oraz integralność. W projekcie zdefiniowano następujące klasy:

- Employee
- LeaveRequest
- AttendanceRecord
- PerformanceReview

Employee

Model przechowuje informacje o pracowniku i jest powiązany relacją 1-1 z kontem użytkownika (User). Ważne pola: stanowisko (position), telefon (phone_number), data zatrudnienia (hire_date) oraz flaga aktywności.

LeaveRequest

Model przechowuje wnioski urlopowe. Zawiera datę początku i końca urlopu, powód oraz status. Własność days oblicza liczbę dni urlopu na podstawie start_date i end_date.

W modelu zastosowano ograniczenie integralności CheckConstraint zapewniające, że end_date >= start_date. W nowszych wersjach Django należy używać argumentu condition w CheckConstraint.

25:4260385107

AttendanceRecord

Model rejestruje obecność pracownika w danym dniu. Przechowuje status (np. Obecny, Zdalnie, Urlop, Nieobecny), liczbę przepracowanych minut oraz notatki (notes) umożliwiające dopisanie informacji kontekstowych.

PerformanceReview

Model przechowuje miesięczne podsumowanie frekwencji (rok i miesiąc) dla danego pracownika. Metoda recompute_from_attendance przelicza wartości working_days i absent_days na podstawie zapisów AttendanceRecord. Własność attendance_score zwraca frekwencję w procentach.

5.1.3 Serializery (core/serializers.py)

Serializery odpowiadają za konwersję obiektów ORM do formatu JSON oraz walidację danych wejściowych. W projekcie zastosowano m.in.:

• UserSerializer – prezentacja podstawowych danych użytkownika (email, imię, nazwisko).

• EmployeeSerializer – odczyt danych pracownika dla list i podglądu.

• EmployeeManageSerializer – tworzenie i edycja pracownika oraz konta użytkownika; obsługa opcjonalnego hasła.

• LeaveRequestSerializer – walidacja dat oraz wyliczanie liczby dni (get_days).

• AttendanceRecordSerializer – zapis/odczyt obecności wraz z notatkami.

• PerformanceReviewSerializer – prezentacja danych oraz frekwencji procentowej.

5.1.4 Widoki i logika biznesowa (core/views.py)

Logika API została zrealizowana w postaci ViewSetów. Wprowadzono kontrolę dostępu oraz rozdzielenie logiki dla użytkownika zwykłego i administratora/menedżera.

IsAdminOrReadOwn

Klasa uprawnień pozwala administratorowi na pełny dostęp, natomiast zwykłemu użytkownikowi jedynie na odczyt własnych danych. Metody has_permission i has_object_permission sprawdzają, czy użytkownik jest zalogowany oraz czy obiekt należy do niego.

EmployeeViewSet

ViewSet obsługuje pracowników. Metody get_queryset i get_serializer_class dobierają zakres danych oraz serializer. Dodatkowe akcje umożliwiają:

26:2873203934

• create_with_user – utworzenie pracownika wraz z kontem użytkownika w jednej operacji;

• update_user – aktualizacja danych konta użytkownika (email, imię, nazwisko);

• set_password – opcjonalna zmiana hasła dla pracownika.

LeaveRequestViewSet

ViewSet obsługuje wnioski urlopowe. get_queryset zwraca wnioski użytkownika lub wszystkie (dla admina). Akcje approve oraz reject zmieniają status wniosku na zaakceptowany lub odrzucony.

AttendanceRecordViewSet

ViewSet obsługuje rejestr obecności. Oprócz standardowego CRUD posiada akcję import_csv, która przyjmuje plik CSV i tworzy rekordy AttendanceRecord. Walidowane są kolumny wejściowe oraz powiązanie pracownika po adresie email.

PerformanceReviewViewSet

ViewSet obsługuje miesięczne podsumowania frekwencji. Akcja export generuje raport w wybranym formacie: CSV, XLSX lub PDF. PDF tworzony jest po stronie serwera (reportlab) i zawiera tabelę z wynikami. [11]

5.1.5 Routing i integracja SPA (core/urls.py, hr_manager/urls.py)

W plikach urls.py zdefiniowano ścieżki dla API (prefiks /api/) oraz endpointy uwierzytelniania. Dodatkowo skonfigurowano regułę catch-all, która przekazuje nie-API ścieżki do pliku index.html, aby routing po stronie React działał również po odświeżeniu strony.

27:3285374356

5.1.6 Uwierzytelnianie sesyjne i ochrona CSRF

Aplikacja korzysta z uwierzytelniania opartego o sesje (cookies). W przypadku SPA konieczne jest dodatkowo przekazywanie tokenu CSRF. Frontend pobiera cookie CSRF i dołącza nagłówek X-CSRFToken w żądaniach modyfikujących dane. [12]

Listing 5.1. Fragment konfiguracji CSRF oraz przykład nagłówka w zapytaniu.

a) hr_manager/hr_manager/settings.py

```
CORS_ALLOWED_ORIGINS = _env_list(
    "DJANGO_CORS_ALLOWED_ORIGINS",
    [
        "http://localhost:5173",
        "http://127.0.0.1:5173",
    ],
)
CORS_ALLOW_CREDENTIALS = True

CSRF_TRUSTED_ORIGINS = _env_list(
    "DJANGO_CSRF_TRUSTED_ORIGINS",
    [
        "http://localhost:5173",
        "http://127.0.0.1:5173",
        "http://localhost:8000",
        "http://127.0.0.1:8000",
    ],
)
```

b) hr_manager/core/views.py

```
@api_view(["GET"])
@permission_classes([permissions.AllowAny])
@ensure_csrf_cookie
def csrf_token(request):
    """Ustawia ciasteczko CSRF dla przeglądarki (wymagane przy żądaniach POST/PUT/PATCH/DELETE w SPA)."""
    return Response({"csrftoken": get_token(request)})
```


28:7550169877

c) Przykład nagłówka w zapytaniu

```
headers: {
  "X-Requested-With": "XMLHttpRequest",
  "X-CSRFToken": csrftoken
}
```

5.1.7 Walidacja wniosków urlopowych

Walidacja obejmuje poprawność zakresu dat, wykrywanie konfliktów z innymi wnioskami oraz kontrolę limitu urlopu w ujęciu rocznym (również dla przełomu roku).

Listing 5.2. Fragment walidacji w serializerze LeaveRequestSerializer.

(hr_manager/core/serializers.py

```
def validate(self, attrs):
    """
    Walidacja wniosku urlopowego (backend):
    1) Daty muszą być poprawne (end >= start)
    2) Urlop musi obejmować co najmniej 1 dzień roboczy (pon–pt)
    3) Zakres nie może kolidować z innym wnioskiem pracownika (pending/approved)
    4) Limit roczny dni urlopu (domyślnie 26) – liczony osobno dla każdego roku,
       również z uwzględnieniem wniosków oczekujących jako „zarezerwowane”.
    """
    start = attrs.get("start_date") or getattr(self.instance, "start_date", None)
    end = attrs.get("end_date") or getattr(self.instance, "end_date", None)

    if start and end and end < start:
        raise serializers.ValidationError({"end_date": "End date must be greater than or
equal to start date."})

    # Ustalamy pracownika:
    emp = attrs.get("employee") or getattr(self.instance, "employee", None)
    if emp is None:
        req = self.context.get("request")
        user = getattr(req, "user", None) if req else None
        if user and user.is_authenticated:
```

```
        emp = Employee.objects.filter(user=user).first()

    if emp is None or not start or not end:
        return attrs

    # (2) Urlop musi mieć co najmniej 1 dzień roboczy
    total_req_days = count_workdays(start, end)
    if total_req_days <= 0:
        raise serializers.ValidationError({"detail": "Urlop musi obejmować co najmniej 1
dzień roboczy (pon–pt)."})

    # (3) Brak kolizji z innymi wnioskami (pending/approved) tego pracownika
    conflicts = (
        LeaveRequest.objects.filter(employee=emp)
        .exclude(status=LeaveRequest.STATUS_REJECTED)
        .filter(start_date__lte=end, end_date__gte=start)
    )
    if self.instance and getattr(self.instance, "pk", None):
        conflicts = conflicts.exclude(pk=self.instance.pk)

    if conflicts.exists():
        c = conflicts.order_by("start_date", "id").first()
        raise serializers.ValidationError(
            {"detail": (
                "Zakres urlopu koliduje z innym wnioskiem urlopowym tego pracownika "
                f"({c.start_date} – {c.end_date}, status: {c.status})."
            )}
        )

    limit_days = settings.HR_DEFAULT_ANNUAL_LEAVE_LIMIT_DAYS

    def workdays_in_year(s: date, e: date, year: int) -> int:
        ys = date(year, 1, 1)
        ye = date(year, 12, 31)
```

```
        s2 = max(s, ys)
        e2 = min(e, ye)
        if e2 < s2:
            return 0
        return count_workdays(s2, e2)

    def used_workdays(year: int) -> int:
        ys = date(year, 1, 1)
        ye = date(year, 12, 31)
        qs = (
            LeaveRequest.objects.filter(
                employee=emp,
                status__in=[LeaveRequest.STATUS_PENDING,
LeaveRequest.STATUS_APPROVED],
                start_date__lte=ye,
                end_date__gte=ys,
            )
            .exclude(status=LeaveRequest.STATUS_REJECTED)
        )
        if self.instance and getattr(self.instance, "pk", None):
            qs = qs.exclude(pk=self.instance.pk)

        total = 0
        for lr in qs.distinct():
            total += workdays_in_year(lr.start_date, lr.end_date, year)
        return total

    # (4) Limit roczny liczony osobno dla każdego roku w zakresie urlopu
    for y in range(start.year, end.year + 1):
        req_days = workdays_in_year(start, end, y)
        if req_days <= 0:
            continue
        used = used_workdays(y)
        if used + req_days > limit_days:
```


31:6954558772

```
            remaining = max(limit_days - used, 0)
            raise serializers.ValidationError(
                {"detail": (
                    f"Brak wystarczającej liczby dni urlopu w roku {y}. "
                    f"Pozostało: {remaining}, próbujesz zarezerwować: {req_days}."
                )}
            )

        return attrs
```

5.1.8 Import obecności z CSV

Import CSV realizowany jest przez dedykowany endpoint API. Każdy wiersz pliku jest walidowany niezależnie, a system zwraca raport z liczbą rekordów utworzonych, zaktualizowanych i błędnych.

Listing 5.3. Fragment widoku importu CSV oraz format oczekiwanych kolumn. (hr_manager/core/views.py)

```
@action(
    detail=False,
    methods=["post"],
    url_path="import",
    permission_classes=[permissions.IsAdminUser],
)
def import_csv(self, request):
    """
    Import CSV: email,date,status,minutes,notes
    Endpoint: /api/attendance/import/
    """
    file = request.FILES.get("file")
    if not file:
        return Response({"detail": "No file provided."}, status=400)

    try:
```


32:6064885201

```
    text = io.TextIOWrapper(file.file, encoding="utf-8")
except Exception:
    return Response({"detail": "Cannot read file."}, status=400)

import csv as _csv

reader = _csv.DictReader(text)
required = {"email", "date", "status", "minutes", "notes"}
if reader.fieldnames is None or not set(reader.fieldnames).issuperset(required):
    return Response(
        {"detail": "CSV must have columns: email,date,status,minutes,notes"},
        status=400,
    )

created = 0
updated = 0
errors = []

allowed_statuses = {k for k, _ in AttendanceRecord.STATUS_CHOICES}

for i, row in enumerate(reader, start=2):
    email = (row.get("email") or "").strip()
    d = (row.get("date") or "").strip()
    status_val = (row.get("status") or "").strip()
    minutes_raw = ((row.get("minutes_worked") or row.get("minutes")) or "").strip()
    notes = (row.get("notes") or "").strip()

    if not email or not d:
        errors.append({"row": i, "message": "missing email/date"})
        continue

    # status validation (to match model choices)
    if not status_val:
        status_val = AttendanceRecord.STATUS_PRESENT
```

```
        if status_val not in allowed_statuses:
            errors.append(
                {"row": i,
                 "message": f"invalid status: {status_val} (allowed: {',
'.join(sorted(allowed_statuses))})"}
            )
            continue

        # minutes validation (prevent 500 on negative / non-number)
        if minutes_raw == "":
            minutes_int = 0
        else:
            try:
                minutes_int = int(float(minutes_raw))
            except ValueError:
                errors.append({"row": i, "message": f"minutes is not a number: {minutes_raw}"})
                continue

        if minutes_int < 0:
            errors.append({"row": i, "message": f"minutes must be >= 0: {minutes_int}"})
            continue

        # notes length safety (avoid DB errors)
        if len(notes) > 255:
            errors.append({"row": i, "message": f"notes too long (max 255), got {len(notes)}"})
            continue

        try:
            emp = Employee.objects.select_related("user").get(user__email=email)
        except Employee.DoesNotExist:
            errors.append({"row": i, "message": f"employee not found: {email}"})
            continue

        try:
```


34:8976106859

```
            rec_date = dt_date.fromisoformat(d)
        except ValueError:
            errors.append({"row": i, "message": f"bad date: {d}"})
            continue

        # Save + recompute must not crash the whole import
        try:
            _, was_created = AttendanceRecord.objects.update_or_create(
                employee=emp,
                date=rec_date,
                defaults={
                    "status": status_val,
                    "minutes_worked": minutes_int,
                    "notes": notes,
                },
            )
            if was_created:
                created += 1
            else:
                updated += 1

            try:
                PerformanceReview.recompute_from_attendance(emp, rec_date.year,
rec_date.month)
            except Exception as e:
                errors.append({"row": i, "message": f"saved, but recompute failed: {e}"})

        except Exception as e:
            errors.append({"row": i, "message": f"save failed: {e}"})
            continue

    return Response(
        {
            "created": created,
```

```
        "updated": updated,
        "error_rows": len(errors),
        "errors": errors,
    }
)
```

5.1.9 Eksport raportów (CSV/XLSX/PDF)

Eksport danych zrealizowano po stronie backendu. Dane są przygotowywane tabelarycznie i serializowane do formatu CSV, XLSX lub PDF. W PDF stosowany jest prosty układ tabeli. [9] [10] [11]

Listing 5.4. Fragment generowania eksportu i ustawienia nagłówków odpowiedzi. (hr_manager/core/views.py)

```
    @action(detail=False, methods=["get"], url_path="export")
def export(self, request):
    fmt = (request.query_params.get("file") or "csv").lower()
    qs = self.get_queryset()
    data = [
        {
            "id": r.id,
            "employee": r.employee.user.email,
            "year": r.year,
            "month": r.month,
            "working_days": r.working_days,
            "absent_days": r.absent_days,
            "attendance_score": r.attendance_score,
            "comments": r.comments or "",
        }
        for r in qs
    ]
    df = pd.DataFrame(data)

    if fmt == "xlsx":
```


36:1945369163

```
        buf = io.BytesIO()
        with pd.ExcelWriter(buf, engine="openpyxl") as writer:
            df.to_excel(writer, index=False, sheet_name="reviews")
        buf.seek(0)
        return FileResponse(
            buf,
            as_attachment=True,
            filename="performance.xlsx",
            content_type="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        )

    if fmt == "pdf":
        try:
            from reportlab.lib import colors
            from reportlab.lib.pagesizes import A4
            from reportlab.lib.styles import getSampleStyleSheet
            from reportlab.platypus import Paragraph, SimpleDocTemplate, Spacer, Table,
TableStyle

            buf = io.BytesIO()
            doc = SimpleDocTemplate(buf, pagesize=A4)
            styles = getSampleStyleSheet()
            story = [Paragraph("Performance Reviews", styles["Heading2"]), Spacer(1, 10)]

            headers = ["ID", "Email", "Rok", "Mies.", "Robocze", "Nieob.", "Frekw.%",
"Komentarz"]
            table_data = [headers] + [
                [
                    row["id"],
                    row["employee"],
                    row["year"],
                    row["month"],
                    row["working_days"],
                    row["absent_days"],
```

```
            row["attendance_score"],
            row["comments"],
        ]
        for row in data
    ]

    table = Table(table_data, repeatRows=1)
    table.setStyle(
        TableStyle(
            [
                ("BACKGROUND", (0, 0), (-1, 0), colors.lightgrey),
                ("GRID", (0, 0), (-1, -1), 0.25, colors.grey),
                ("ALIGN", (0, 0), (-1, -1), "CENTER"),
                ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
            ]
        )
    )
    story.append(table)
    doc.build(story)
    buf.seek(0)
    return FileResponse(
        buf,
        as_attachment=True,
        filename="performance.pdf",
        content_type="application/pdf",
    )
  except Exception:
    pass  # fallback to CSV below

csv_data = df.to_csv(index=False).encode("utf-8")
resp = HttpResponse(csv_data, content_type="text/csv; charset=utf-8")
resp["Content-Disposition"] = f'attachment; filename="{smart_str("performance.csv")}"'
return resp
```

5.1.10 Uwagi dotyczące bazy danych i integralności

W modelach zastosowano ograniczenia integralności (constraints) oraz indeksy, które poprawiają wydajność wyszukiwania i zapobiegają duplikacji danych, np. unikalność obecności dla pary (employee, date).

5.2 Frontend – React i Tailwind CSS

Frontend został wykonany jako aplikacja SPA w React (Vite) z wykorzystaniem Tailwind CSS do stylowania. Komunikacja z backendem odbywa się przez żądania HTTP do REST API, a stan zalogowania jest przechowywany w kontekście (AuthContext). [3] [4] [5]

5.2.1 Struktura projektu (frontend/src)

• main.jsx – punkt wejścia aplikacji.

• App.jsx – routing (react-router-dom) oraz ochrona tras wymagających logowania.

• context/AuthContext.jsx – logika logowania, przechowywanie tokenu i danych użytkownika.

• http.js / api.js – konfiguracja klienta HTTP i funkcje ułatwiające wywołania API.

• pages/* – implementacje widoków: Login, Dashboard, Pracownicy, Urlopy, Obecność, Oceny.

• components/* – współdzielone komponenty UI (badge, statystyki, przełącznik wykresów).

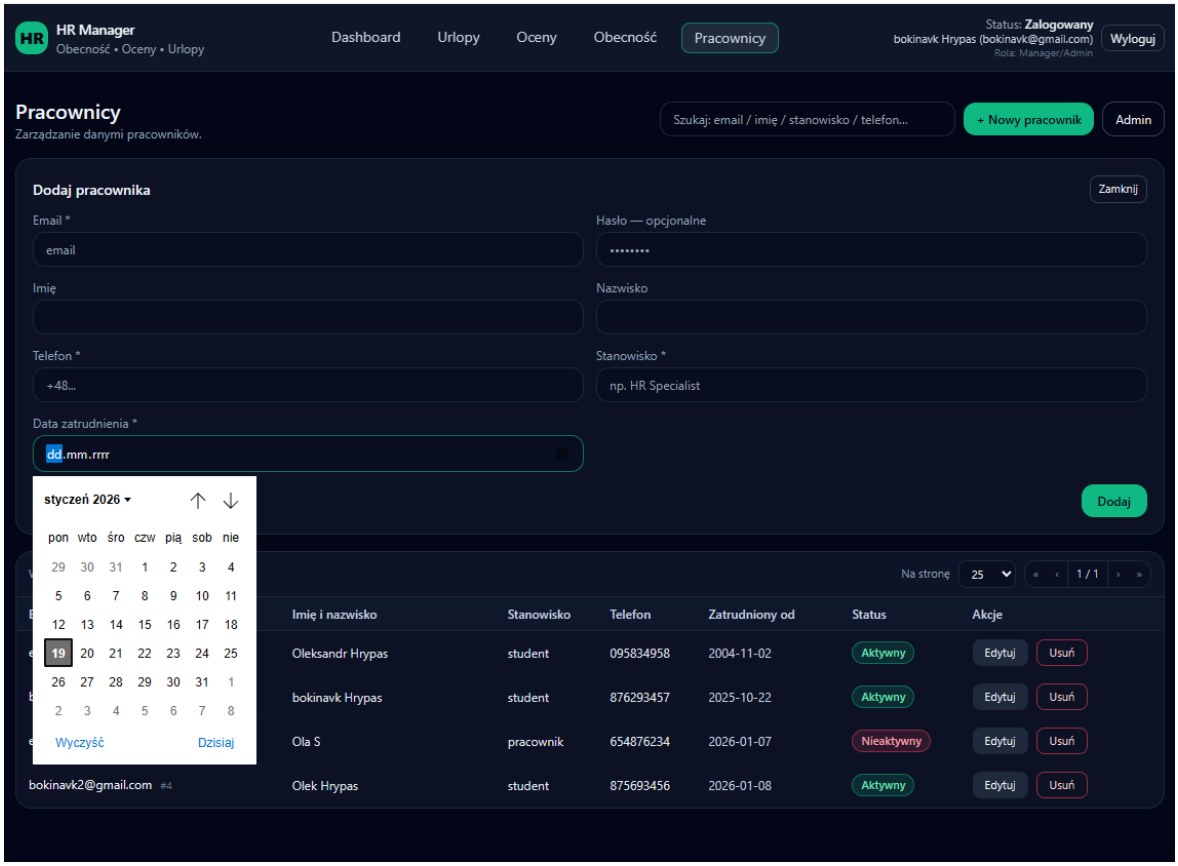
5.2.2 Uwierzytelnianie i autoryzacja (AuthContext)

Moduł AuthContext przechowuje informacje o użytkowniku oraz token i udostępnia metody logowania i wylogowania. Po zalogowaniu interfejs pobiera dane z endpointu /api/me/ i wyświetla je w nagłówku (status zalogowania oraz rola).

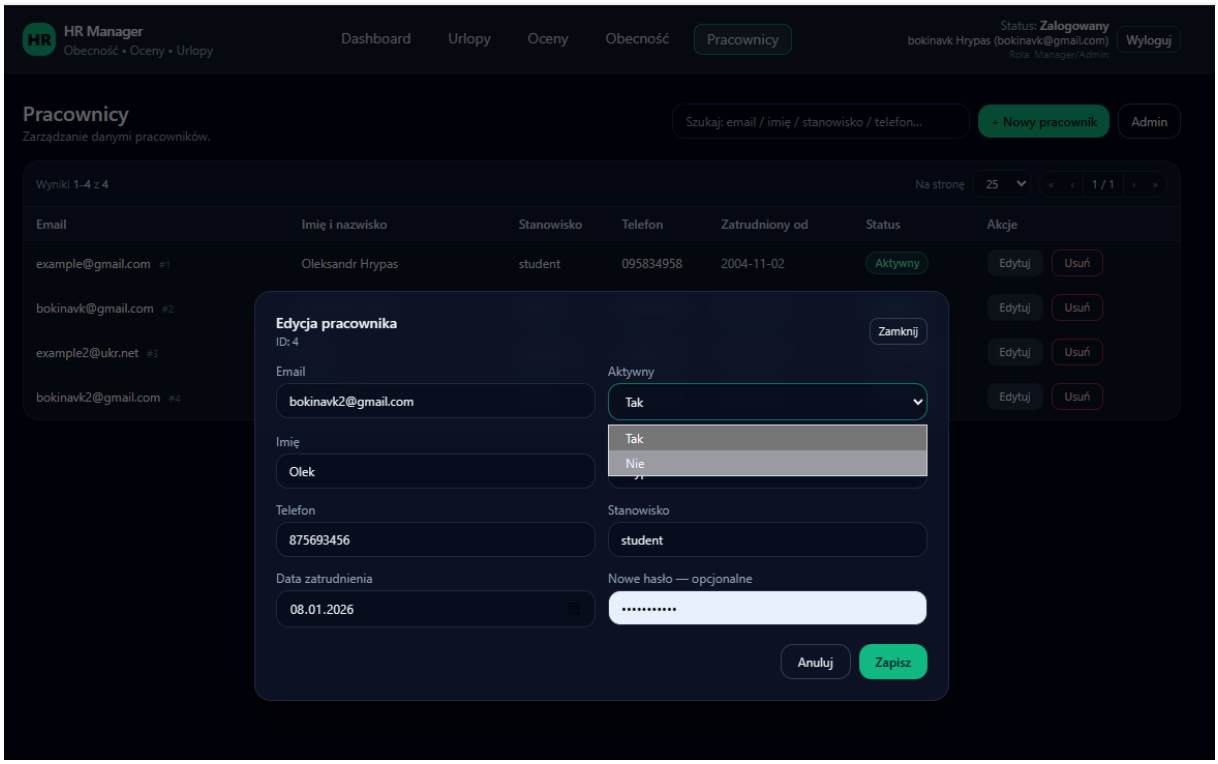
5.2.3 Widoki aplikacji (pages/*)

Każdy widok realizuje pobieranie danych z API, obsługę formularzy oraz aktualizację tabel. Poniżej krótki opis najważniejszych ekranów:

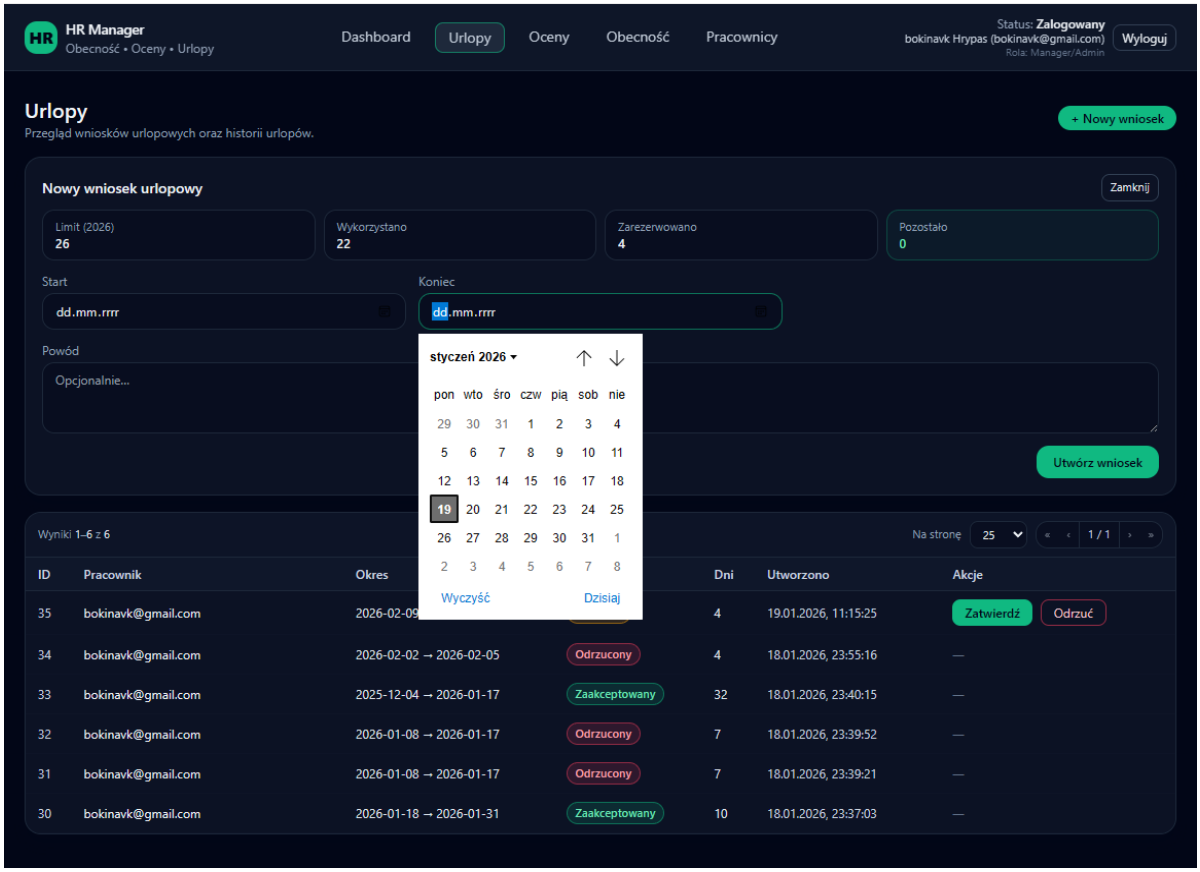
39:9602624937



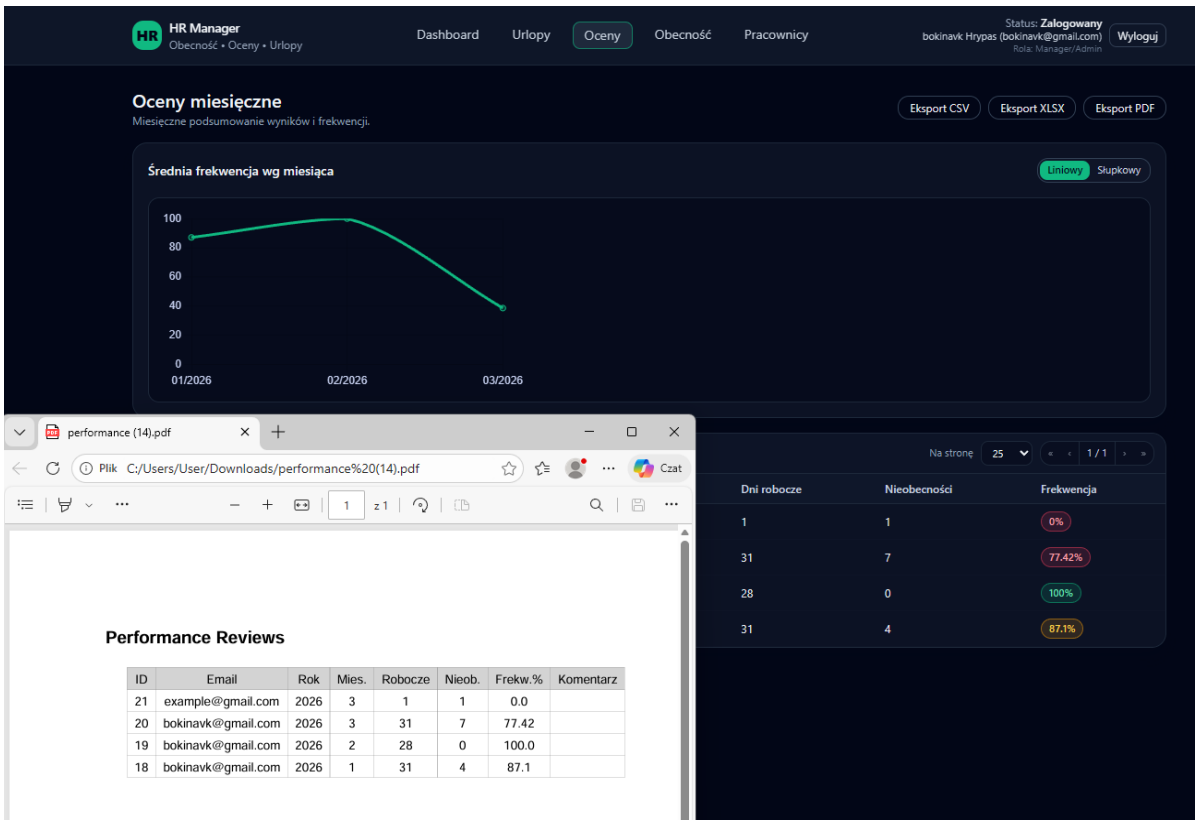
Rys. 5.1. Formularz dodawania pracownika.



Rys. 5.2. Okno edycji danych pracownika.



Rys. 5.3. Formularz tworzenia wniosku urlopowego (limit/wykorzystano/pozostało).



Rys. 5.5. Wynik eksportu PDF w module Oceny.

Pracownicy: widok listy pracowników z wyszukiwaniem oraz paginacją. Operacje „Dodaj”, „Edytuj” i „Usuń” wywołują odpowiednie endpointy /api/employees/.
Urlopy: widok umożliwia utworzenie wniosku urlopowego oraz przegląd jego statusu.
Obecność: widok listy wpisów dziennych; zawiera import CSV i możliwość dopisywania notatek.
Oceny: widok prezentuje miesięczne podsumowania frekwencji wraz z wykresem oraz eksportem raportu.

5.2.4 Tabele, paginacja i praca z dużą liczbą rekordów

W modułach, w których liczba rekordów może być duża (obecność, pracownicy, urlopy, oceny) zastosowano paginację oraz kontenery przewijania. Dzięki temu tabela pozostaje czytelna, a interfejs działa płynnie również dla większych zbiorów danych.

5.2.5 Panel administracyjny Django

Django Admin jest wykorzystywany jako narzędzie administracyjne do przeglądania i edycji danych w bazie. W panelu widoczne są modele aplikacji core oraz standardowe modele Django/allauth.

5.2.6 Warstwa HTTP i obsługa CSRF w SPA

Zastosowano wspólny moduł komunikacji HTTP, który odpowiada za dołączanie nagłówków oraz obsługę błędów. Dla żądań modyfikujących dane automatycznie dołączany jest token CSRF.

Listing 5.5. Fragment pliku http.js – ustawianie X-CSRFToken. (hr_manager/frontend/src/http.js)

```
export function getCookie(name) {
  const value = `; ${document.cookie}`;
  const parts = value.split(`; ${name}=`);
  if (parts.length === 2) return parts.pop().split(";").shift();
  return null;
}

export async function apiPost(path, payload = null) {
  const url = `${BACKEND_BASE}${path}`;
  await ensureCsrfCookie();
  const csrftoken = getCookie("csrftoken");
```

```
  const res = await fetch(url, {
    method: "POST",
    credentials: "include",
    headers: {
      "X-Requested-With": "XMLHttpRequest",
      ...(csrftoken ? { "X-CSRFToken": csrftoken } : {}),
      ...(payload ? { "Content-Type": "application/json" } : {}),
    },
    body: payload ? JSON.stringify(payload) : null,
  });

  if (!res.ok) {
    let msg = `HTTP ${res.status}`;
    try {
      const j = await res.json();
      msg = j?.detail || msg;
    } catch {
      // ignore
    }
    throw new ApiError(msg, res.status);
  }

  try {
    return await res.json();
  } catch {
    return {};
  }
}

// ---------------- CSRF + JSON REQUEST (PATCH/DELETE/PUT) ----------------

export async function apiRequest(method, path, payload = null) {
  const url = `${BACKEND_BASE}${path}`;
  await ensureCsrfCookie();
```

```
  const csrftoken = getCookie("csrftoken");

  const isFormData =
    typeof FormData !== "undefined" && payload instanceof FormData;

  const res = await fetch(url, {
    method,
    credentials: "include",
    headers: {
      "X-Requested-With": "XMLHttpRequest",
      ...(csrftoken ? { "X-CSRFToken": csrftoken } : {}),
      ...(payload && !isFormData ? { "Content-Type": "application/json" } : {}),
    },
    body: payload ? (isFormData ? payload : JSON.stringify(payload)) : null,
  });

  if (!res.ok) {
    let msg = `HTTP ${res.status}`;
    try {
      const j = await res.json();
      msg = j?.detail || msg;
    } catch {
      // ignore
    }
    throw new ApiError(msg, res.status);
  }

  // DELETE może zwrócić pustą odpowiedź
  try {
    return await res.json();
  } catch {
    return {};
  }
}
```

```
export function apiPatch(path, payload) {
  return apiRequest("PATCH", path, payload);
}

export function apiDelete(path) {
  return apiRequest("DELETE", path);
}

export async function ensureCsrfCookie() {
  // Backend ustawia cookie CSRF; bez tego POST/PUT/PATCH/DELETE mogą zwracać
403.
  if (getCookie("csrftoken")) return;

  try {
    await fetch(`${BACKEND_BASE}/api/csrf/`, {
      method: "GET",
      credentials: "include",
      headers: { "X-Requested-With": "XMLHttpRequest" },
    });
  } catch {
    // w trybie offline/bez backendu po prostu pomijamy
  }
}
```

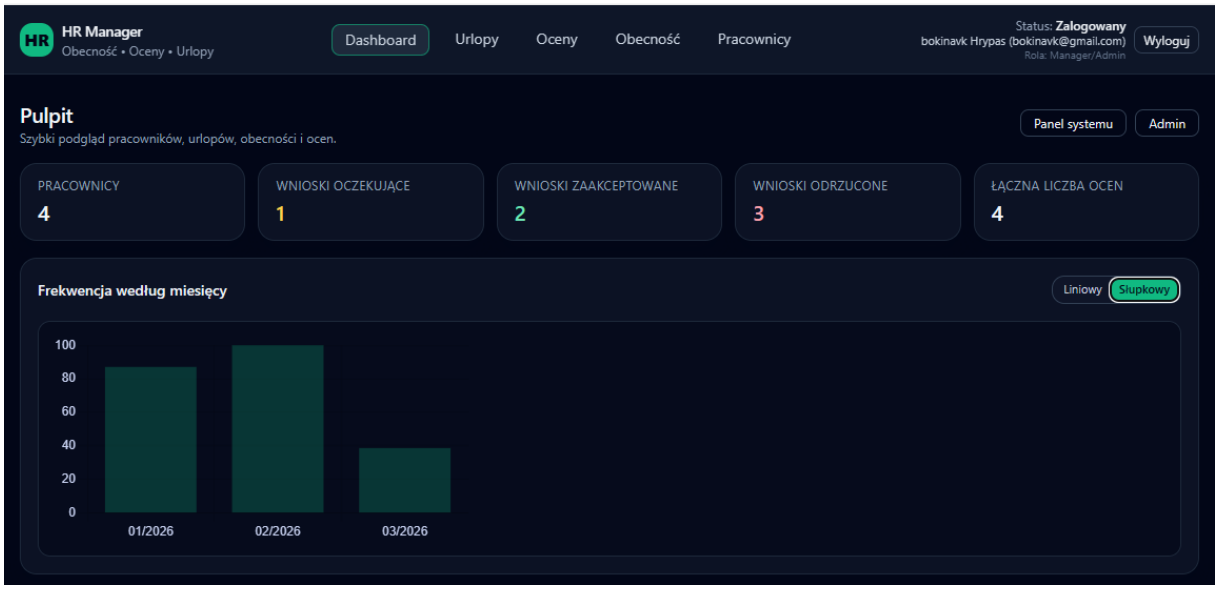
5.2.7 Komponenty reużywalne (tabele, badge, formularze)

W celu ograniczenia duplikacji kodu zastosowano komponenty reużywalne, m.in. tabelę z paginacją, komponenty statusów (badge) oraz wspólne układy formularzy.

5.2.8 Wizualizacja danych (Chart.js)

W module ocen miesięcznych zastosowano wykresy prezentujące dane frekwencyjne. [6]

45:7464880655



Rys. 5.8. Wykres frekwencji w panelu – zrzut ekranu.

5.2.9 Obsługa błędów i komunikaty użytkownika

Interfejs prezentuje komunikaty błędów zwracane przez API (np. walidacja urlopu lub błędy importu).

5.2.10 Wydajność i ergonomia UI

Zastosowano paginację list oraz ograniczono ilość danych pobieranych jednorazowo. Interfejs zaprojektowano tak, aby typowe operacje wymagały minimalnej liczby kliknięć.

5.3 Struktura repozytorium i organizacja kodu

Repozytorium projektu podzielono na dwie główne części: backend (Django/DRF) oraz frontend (React). Dzięki temu możliwy jest niezależny rozwój warstw oraz czytelne rozdzielenie odpowiedzialności.

Przykładowa struktura katalogów:

hr_manager/ - konfiguracja projektu Django (settings, urls)

core/ - logika domenowa (modele, serializery, widoki, admin)

frontend/ - aplikacja SPA (Vite/React)

frontend/src/ - komponenty, strony, logika UI

requirements.txt - zależności backendu

package.json - zależności frontendu

5.4 Dokumentacja endpointów API

Tabela przedstawia kluczowe endpointy API wraz z metodą, przeznaczeniem oraz wymaganym poziomem uprawnień. Endpointy zostały zaprojektowane w stylu REST.

Metoda	Endpoint	Rola	Opis
POST	/accounts/login/	wszyscy	Logowanie (sesja).
GET	/accounts/logout/	zalogowany	Wylogowanie.
GET	/api/me/	zalogowany	Dane zalogowanego użytkownika.
GET	/api/employees/	admin/menedżer	Lista pracowników.
POST	/api/employees/	admin	Utworzenie pracownika.
GET	/api/employees/{id}/	admin/menedżer	Szczegóły pracownika.
PATCH	/api/employees/{id}/	admin	Edycja pracownika.
GET	/api/leave-requests/	zalogowany	Lista wniosków (własne lub wszystkie).
POST	/api/leave-requests/	pracownik	Złożenie wniosku.
POST	/api/leave-requests/{id}/approve/	admin/menedżer	Akceptacja wniosku.
POST	/api/leave-requests/{id}/reject/	admin/menedżer	Odrzucenie wniosku.
GET	/api/attendance/	admin/menedżer	Lista obecności.
POST	/api/attendance/	admin/menedżer	Dodanie/edycja obecności.
POST	/api/attendance/import/	admin	Import CSV.
GET	/api/performance/	admin/menedżer	Podsumowania miesięczne.
GET	/api/performance/export/?file=csv|xlsx|pdf	admin/menedżer	Eksport raportu.

5.5 Opis bazy danych (tabele, relacje, indeksy)

Baza danych zawiera zestaw tabel odpowiadających modelom domenowym. Poniżej opisano najważniejsze tabele oraz relacje pomiędzy nimi.

Najważniejsze tabele:

• auth_user – tabela użytkowników (Django).

• core_employee – profil pracownika powiązany z użytkownikiem.

• core_leaverequest – wnioski urlopowe wraz ze statusem.

• core_attendancerecord – dzienne wpisy obecności.

47:5841357847

• core_performancereview – miesięczne podsumowania i wskaźniki.

Relacje: core_employee (1)–(1) auth_user; core_leaverequest/attendance/performance (N)–(1) core_employee.

Indeksy i ograniczenia: unikalność (employee_id, date) dla obecności, indeksy po pracowniku i dacie, ograniczenia na statusy i zakresy (minutes >= 0).

Diagram ERD bazy danych – zob. Rys. 4.2.

6. Testowanie

Testowanie wykonano w formie testów manualnych (scenariusze użytkownika) w środowisku lokalnym. Tabela 6.1 zawiera przykładowe scenariusze testowe.

Tabela 6.1. Scenariusze testów manualnych.

ID	Moduł	Kroki (skrót)	Oczekiwany rezultat
T-01	Logowanie	Poprawne dane	Zalogowanie, dostęp do panelu
T-02	Logowanie	Błędne hasło	Komunikat błędu, brak sesji
T-03	Uprawnienia	Wejście w moduł admin jako pracownik	403 / brak akcji
T-06	Urlopy	start > end	Błąd walidacji
T-07	Urlopy	Weekend	Błąd: 1 dzień roboczy
T-08	Urlopy	Konflikt	Błąd konfliktu
T-09	Urlopy	Limit	Błąd limitu
T-10	Urlopy	Approve	Status approved
T-11	Urlopy	Reject	Status rejected
T-14	Import CSV	Brak pliku	400
T-15	Import CSV	Brak kolumn	400
T-16	Import CSV	Błędny wiersz	Błąd per wiersz, kontynuacja
T-20	Eksport	CSV/XLSX/PDF	Poprawny plik

Scenariusz	Kroki	Oczekiwany wynik
Logowanie	Podaj poprawne i błędne dane logowania.	Poprawne dane logują; błędne zwracają komunikat.
CRUD pracowników	Dodaj pracownika, edytuj dane, usuń rekord.	Rekordy są poprawnie tworzone/aktualizowane/usuwane.
Wnioski urlopowe	Utwórz wniosek; zmień status przez menedżera.	Status zmienia się na zaakceptowany/odrzucony.
Import CSV	Wczytaj plik CSV	Wpisy pojawiają się w

	obecności.	tabeli, błędne wiersze są pomijane z komunikatem.
Eksport	Wygeneruj raport CSV/XLSX/PDF.	Plik jest generowany i zawiera dane zgodne z tabelą.

Testowanie przeprowadzono w środowisku lokalnym. Obejmowało ono testy manualne interfejsu, walidacji serwerowej (API) oraz scenariusze negatywne. Dodatkowo wykonano testy uprawnień w celu weryfikacji izolacji danych pomiędzy użytkownikami.

6.1 Plan testów

Plan testów obejmował: testy funkcjonalne (CRUD, logowanie, urlopy, obecności, eksport), testy walidacji danych oraz testy uprawnień.

6.2 Testy API (walidacja backendu)

Testy API wykonano poprzez wysyłanie żądań HTTP do endpointów REST i weryfikację kodów odpowiedzi (200/201/400/403) oraz treści komunikatów walidacyjnych.

6.3 Testy walidacji wniosków urlopowych

Zweryfikowano odrzucanie zakresu, gdy end_date < start_date, odrzucanie wniosków bez dni roboczych, wykrywanie konfliktów oraz kontrolę limitu rocznego (także przy przełomie roku).

6.4 Testy bezpieczeństwa i uprawnień

Sprawdzono, że pracownik nie ma dostępu do operacji administracyjnych, a endpointy importu i eksportu wymagają odpowiedniej roli. Zweryfikowano również działanie ochrony CSRF w SPA.

6.5 Testy interfejsu (frontend)

Wykonano testy formularzy, paginacji i wyszukiwania w tabelach, poprawności nawigacji oraz responsywności.

6.6 Testy wydajnościowe (podstawowe)

Oceniono zachowanie list przy większej liczbie rekordów (paginacja), czas importu CSV o większej liczbie wierszy oraz generowanie eksportów.

6.7 Wnioski z testów

Wyniki testów potwierdziły poprawność realizacji założonych funkcji. Import CSV zaprojektowano tak, aby błędne wiersze nie blokowały całego procesu, co zwiększa odporność systemu w praktyce.

7. Instrukcja uruchomienia

7.1 Wymagania środowiskowe

Wersje użyte w projekcie: Python 3.12.10, Django 4.2+, Node.js v24.11.1, npm 11.6.2.

7.2 Uruchomienie backendu (Windows)

- Utwórz środowisko wirtualne: python -m venv .venv oraz aktywuj je.
- Zainstaluj zależności: pip install -r requirements.txt
- Wykonaj migracje: python manage.py migrate
- Utwórz konto administratora: python manage.py createsuperuser
- Uruchom serwer: python manage.py runserver

7.3 Uruchomienie frontendu

- Przejdź do katalogu frontend: cd frontend
- Zainstaluj zależności: npm install
- Uruchom tryb developerski: npm run dev
- Opcjonalnie: zbuduj wersję produkcyjną: npm run build (pliki w dist).

7.4 Uwagi dotyczące konfiguracji

W wersji demonstracyjnej ustawienia mogą zawierać DEBUG=True oraz testowy SECRET_KEY. Do wdrożenia produkcyjnego zaleca się przeniesienie SECRET_KEY do pliku .env oraz ustawienie DEBUG=False.

7.5 Uruchomienie backendu (Linux/macOS)

- Utwórz środowisko wirtualne: python3 -m venv .venv oraz aktywuj je (source .venv/bin/activate).
- Zainstaluj zależności: pip install -r requirements.txt
- Migracje: python manage.py migrate
- Konto administratora: python manage.py createsuperuser
- Start: python manage.py runserver

7.6 Budowa wersji produkcyjnej frontendu i integracja z Django

W trybie produkcyjnym frontend jest budowany poleceniem npm run build, a pliki dist są serwowane przez backend. Zastosowano routing „catch-all”, który kieruje nieznane ścieżki do index.html, co zapewnia poprawne działanie React Router.

Listing 7.1. Fragment konfiguracji routingu SPA (urls.py).

```
urlpatterns = [
    path("admin/", admin.site.urls),

    # allauth
    path("accounts/", include("allauth.urls")),

    # API
    path("api/", include("core.urls")),

    # DRF browser login
    path("api-auth/", include("rest_framework.urls")),
]

if settings.DEBUG:
```

```
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
    urlpatterns += [
        re_path(
            r"^assets/(?P<path>.*)$",
            static_serve,
            {"document_root": settings.FRONTEND_DIST_DIR / "assets"},
        )
    ]

urlpatterns += [
    re_path(
        r"^(?!api/|admin/|accounts/|api-auth/|static/|media/|assets/).*",
        never_cache(spa_index),
    ),
]
```

7.7 Wdrożenie produkcyjne (koncepcja)

Do wdrożenia produkcyjnego zalecane jest użycie serwera WSGI (np. Gunicorn) oraz reverse proxy (np. Nginx). Nginx może obsługiwać statyczne zasoby oraz TLS, a ruch dynamiczny przekazywać do aplikacji.

7.8 Kopia zapasowa i migracje

W środowisku lokalnym baza SQLite znajduje się w pliku db.sqlite3. Kopię zapasową można wykonać poprzez skopiowanie pliku bazy. W środowisku produkcyjnym zalecane są regularne backupy i kontrola wersji migracji Django.

7.9 Rozwiązywanie problemów

- Błąd CSRF (403): odśwież stronę, zaloguj się ponownie i sprawdź cookies.
- CORS: sprawdź konfigurację domen/portów w ustawieniach backendu.
- Import CSV: zweryfikuj nagłówki kolumn i format dat (YYYY-MM-DD).
- Eksport PDF: sprawdź instalację bibliotek PDF w środowisku backendu.

8. Instrukcja użytkownika

Niniejszy rozdział opisuje sposób korzystania z aplikacji „HR Manager” z perspektywy użytkownika końcowego.

8.1 Role użytkowników

Pracownik ma dostęp do własnych danych i może składać wnioski urlopowe. Menedżer/administrator ma dostęp do danych globalnych, zatwierdza wnioski i wykonuje import/eksport.

8.2 Logowanie i wylogowanie

Po uruchomieniu aplikacji wprowadź e-mail i hasło oraz kliknij „Zaloguj”. Wylogowanie dostępne jest w menu użytkownika.

8.3 Panel główny (Dashboard)

Panel prezentuje podsumowanie stanu wniosków oraz skróty do modułów. Widoczne elementy zależą od roli.

8.4 Moduł „Pracownicy” (admin/menedżer)

W module można przeglądać listę, dodawać nowe osoby, edytować dane i wyszukiwać rekordy.

8.5 Moduł „Urlopy”

Aby utworzyć wniosek: Urlopy → Nowy wniosek → wybierz daty → Zapisz. System waliduje daty, dni robocze, konflikty i limit.

8.6 Moduł „Obecności”

Moduł pozwala rejestrować status i czas pracy. Dla większej liczby danych dostępny jest import CSV.

8.7 Import CSV (admin)

Import wymaga kolumn: email, date, status, minutes, notes. Po imporcie system zwraca raport (utworzone/zaktualizowane/błędne).

8.8 Oceny/Frekwencja i eksport

W module dostępne są zestawienia miesięczne, wykresy oraz eksport do CSV/XLSX/PDF.

8.9 FAQ

- CSRF 403: odśwież stronę i zaloguj się ponownie.
- Import: sprawdź nagłówki i format dat.
- Brak uprawnień: skontaktuj się z administratorem.

9. Podsumowanie i kierunki rozwoju

Zrealizowany system „HR Manager” spełnia założone wymagania: umożliwia prowadzenie rejestru pracowników, obsługę urlopów, ewidencję obecności z importem CSV oraz generowanie miesięcznych podsumowań frekwencji z możliwością eksportu raportów. W przyszłości aplikacja może zostać rozbudowana m.in. o: bardziej rozbudowany system ról (grupy), dodatkowe raporty roczne, integrację z PostgreSQL oraz wdrożenie produkcyjne.

Najważniejsze rezultaty projektu:

- spójny model danych (pracownicy, urlopy, obecności, podsumowania);
- walidacja urlopów (konflikty, dni robocze, limity roczne);
- import CSV odporny na błędy (raport per wiersz);
- automatyczne przeliczenia podsumowań po zmianie obecności;

• interfejs SPA z tabelami, formularzami i wykresami.

Kierunki rozwoju:

• kalendarz świąt i dni wolnych;

• powiadomienia o decyzjach urlopowych;

• wieloetapowe zatwierdzanie;

• rozszerzone KPI i filtrowanie raportów;

• log audytowy i rozbudowane role.

Bibliografia

1. Django Software Foundation. Django Documentation (wersja 6.0). 2025. Dokumentacja online. Dostęp: 2026-01-13. Dostępna w Internecie: https://docs.djangoproject.com/en/6.0/
2. Encode OSS Ltd. (społeczność Django REST framework). Django REST framework Documentation. bd.. Dokumentacja online. Dostęp: 2026-01-13. Dostępna w Internecie: https://www.django-rest-framework.org/
3. Meta Platforms, Inc. (zespół React). React Documentation (react.dev). bd.. Dokumentacja online. Dostęp: 2026-01-13. Dostępna w Internecie: https://react.dev/learn
4. Evan You i współtwórcy. Vite Guide. bd.. Dokumentacja online. Dostęp: 2026-01-13. Dostępna w Internecie: https://vitejs.dev/guide/
5. Tailwind Labs Inc. Tailwind CSS Documentation. bd.. Dokumentacja online. Dostęp: 2026-01-13. Dostępna w Internecie: https://tailwindcss.com/docs/installation
6. Chart.js Contributors. Chart.js Documentation. bd.. Dokumentacja online. Dostęp: 2026-01-13. Dostępna w Internecie: https://www.chartjs.org/docs/latest/
7. Raymond Penners i społeczność projektu. django-allauth Documentation. bd.. Dokumentacja online. Dostęp: 2026-01-14. Dostępna w Internecie: https://docs.allauth.org/en/latest/
8. Adam Johnson i społeczność projektu. django-cors-headers Documentation. bd.. Dokumentacja online. Dostęp: 2026-01-14. Dostępna w Internecie: https://github.com/adamchainz/django-cors-headers

55:5503150493

9. The pandas development team. pandas Documentation. bd.. Dokumentacja online. Dostęp: 2026-01-14. Dostępna w Internecie: https://pandas.pydata.org/docs/
10. openpyxl Developers. openpyxl Documentation. bd.. Dokumentacja online. Dostęp: 2026-01-14. Dostępna w Internecie: https://openpyxl.readthedocs.io/en/stable/
11. ReportLab. ReportLab User Guide (PDF). bd.. Dokumentacja online. Dostęp: 2026-01-14. Dostępna w Internecie: https://www.reportlab.com/docs/reportlab-userguide.pdf
12. OWASP Foundation. Cross-Site Request Forgery Prevention Cheat Sheet. bd.. Materiał online. Dostęp: 2026-01-14. Dostępny w Internecie: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
13. OWASP Foundation. Authentication Cheat Sheet. bd.. Materiał online. Dostęp: 2026-01-14. Dostępny w Internecie: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

56:1908596074